

C++

top

КОМПЬЮТЕРНАЯ
АКАДЕМИЯ

</>

C++

ПРОГРАММИРОВАНИЕ С ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИИ



J A V A

Урок № 6

Введение в разработку
серверных решений
с использованием
Java, взаимодействие
с источниками данных

Содержание

Серверное программирование	3
Фреймворки и библиотеки	6
Maven.....	6
Tomcat	19
JBoss	26
Spring.....	27
Первое Spring приложение	30
Второе Spring приложение	36
Hibernate.....	40
Понятие сервлета	44
Источники данных.....	59
JDBC	59
Пример использования Hibernate.....	70
Домашнее задание.....	84

Серверное программирование

Вы уже знакомы со структурой веб-приложений и помните, что они состоят из клиентской и серверной частей. Клиентской частью веб-приложения, как правило, является браузер, который генерирует запросы к серверной части приложения, получает от сервера ответы на свои запросы и отображает полученную от сервера информацию в клиентской области браузера.

Серверная часть приложения может быть создана с помощью разных программных технологий. Рассмотрим, как создаются веб-приложения на платформе Java. Платформа Java делится на несколько компонентов, основными из которых являются **SE** (*Standard Edition*) и **EE** (*Enterprise Edition*). До сих пор вы работали в пределах платформы SE. Вы уже видели, что эта платформа позволяет создавать широкий диапазон приложений: консольные, с графическим пользовательским интерфейсом, десктопные, сетевые, многопоточные и т.п. Есть ли у нее предел возможностей? Да. Используя Java SE, нельзя создавать веб-приложения. Для создания веб-приложений необходимо пользоваться платформой Java EE. Поэтому нам надо рассмотреть возможности этой платформы.

Однако сначала вам надо познакомиться с новым термином — сервер приложений (*application server*). Как только начали создаваться клиент-серверные приложения,

перед разработчиками встал вопрос: какую часть приложения больше загружать работой, кто в этой паре (клиент или сервер) должен выполнять больше действий?

Часто ответ диктовался логикой конкретной ситуации. Вы, конечно же, знаете ответ на этот вопрос для случая веб-приложений. В паре браузер — веб-сервер клиентская часть, т.е. браузер, выполняет намного больше работы, чем серверная часть — веб-сервер. Это сделано намеренно, потому, что один веб-сервер должен уметь быстро обслуживать много клиентов, и его загруженность стараются минимизировать насколько это возможно. Даже при таком подходе веб-сервер часто является «узкой частью» приложения. Чтобы разгрузить его еще больше, используют так называемые серверы приложений. Сервер приложений — это ПО, которое располагается где-то рядом с серверной частью приложения и забирает на себя часть запросов клиентов, частично разгружая таким образом сам сервер. Возможно, вы не знаете, что давно известный вам IIS — это совсем не веб-сервер, как думают многие. Это типичный сервер приложений, который помимо функций веб-сервера может выполнять еще целый ряд действий. Например, разворачивать службы, в том числе и WCF службы.

В Java EE серверы приложений используются очень широко. Скоро мы с вами будем говорить о таких утилитах, как [Tomcat](#) или [GlassFish](#), которые являются типичными представителями серверов приложений.

Теперь можно коротко поговорить об основных компонентах Java EE. А более подробный разговор продолжим далее в этом уроке.

Итак, основные компоненты Java EE:

- Для обработки клиентских HTTP запросов Java EE предлагает такой API, как **Servlet**. **Servlet** — это Java класс, умеющий принимать клиентские запросы (*request*), обрабатывать их и отправлять клиенту ответы (*response*). **Servlet** разворачивается и выполняется на сервере приложений — например, на **Tomcat** или **GlassFish**.
- Для создания интерактивных, динамических html страниц используется API *Java Server Pages* (**JSP**). Это своего рода серверный язык Java. JSP используется совместно с **Servlet**.
- Для создания веб-сервисов Java EE предлагает технологию *Enterprise JavaBeans* (**EJB**). Если вам понятно утверждение, что обычное Java приложение состоит из классов, то, по аналогии, можете сейчас запомнить, что веб-сервис состоит из EJB компонентов, которые очень напоминают классы, но более приспособлены для веб-сервисов и потому обладают большим числом преимуществ. EJB изначально поддерживают удаленный доступ, являются многопоточными, поддерживают транзакции, обеспечивают целостность данных и обладают еще рядом полезных свойств.
- Java EE также предлагает собственные API для работы с источниками данных, для работы с форматом JSON, для обработки электронных писем, для работы с REST-службами и другие. О некоторых из этих компонентов мы также поговорим в наших уроках.

Подробно посмотреть структуру платформы Java EE можно на официальном сайте по адресу: <http://www.oracle.com/technetwork/java/javaee/tech/index.html>.

Фреймворки и библиотеки

Maven

Вы уже создали немало Java проектов. При этом, скорее всего, вы особо не задумывались о том, что представляет собой проект, из каких частей он состоит, и как эти части между собой связаны. Всю эту работу за вас выполняет IDE. Пришло время рассмотреть этот процесс подробнее.

Из каких этапов состоит создание проекта? Это компиляция исходных файлов, создание jar-файла, дистрибутива и документации. Вообще говоря, это большой объем работы, особенно в том случае, когда собираемый проект состоит из большого числа файлов. Эта работа выполняется специальными инструментами. Сейчас мы с вами рассмотрим один из них — утилиту **Maven** [мэйвен].

Рассмотрим достоинства этой утилиты. Создание Java проекта с помощью **Maven** выполняется на основании сценария, записанного в специальном xml файле. Этот сценарий не зависит от платформы и позволяет собирать проект в ОС Windows или в ОС Linux без изменения файла сценария. Как правило, создание проекта требует подключения различных сторонних библиотек. В такой ситуации необходимо учитывать, что разные версии таких библиотек могут конфликтовать друг с другом. **Maven** умеет разрешать такие проблемы зависимостей и обеспечивает бесконфликтную работу подключенных библиотек. Формат проекта, собранного с помощью **Maven**, свободно открывается основными

интегрированными средами разработки, что также является большим плюсом этой утилиты. Кроме этого, управление процессом создания проекта выполняется декларативно в XML формате.

Давайте выполним установку этой утилиты и соберем с ее помощью простой Java проект. Для установки Maven перейдите на страницу <http://maven.apache.org/download.cgi> и загрузите дистрибутив:

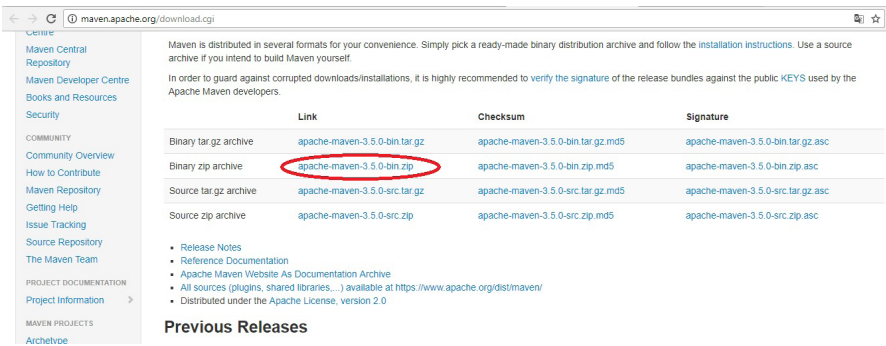


Рис. 1. Загрузка Maven

Теперь распакуйте zip архив в любую директорию на своем диске — например, прямо на системный диск по пути `C:\apache-maven-3.5.0`. Далее нажмите **Win+Pause**, перейдите в «Дополнительные параметры» и в появившемся окне кликните внизу кнопку «Переменные среды». Затем в окне «Системные переменные» создайте следующие переменные с указанными значениями:

- переменную `M2_HOME` со значением пути, куда вы скопировали Maven; в моем случае — `C:\apache-maven-3.5.0`;
- переменную `M2` со значением `%M2_HOME%\bin`;

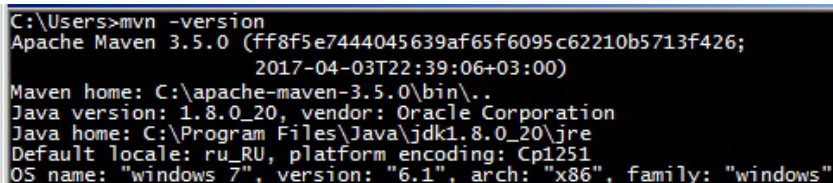
- переменную `MAVEN_OPTS` со значением `-Xms256m` или `-Xms512m`;
- переменную `Path` со значением `%M2%`.

Теперь проверьте, существует ли у вас переменная с именем `JAVA_HOME` и со значением пути, по которому установлен JDK. В моем случае это путь `C:\Program Files\Java\jdk1.8.0_25`. Если этой переменной нет — создайте ее. Перегрузите компьютер.

Выполните в консольном окне команду:

```
mvn -version
```

чтобы убедиться, что установка и настройка **Maven** прошли успешно. Вы должны увидеть версию установленной утилиты и другую сопутствующую информацию. У меня это выглядит так:



```
C:\Users>mvn -version
Apache Maven 3.5.0 (ff8f5e7444045639af65f6095c62210b5713f426;
2017-04-03T22:39:06+03:00)
Maven home: C:\apache-maven-3.5.0\bin\..
Java version: 1.8.0_20, vendor: Oracle Corporation
Java home: C:\Program Files\Java\jdk1.8.0_20\jre
Default locale: ru_RU, platform encoding: Cp1251
OS name: "windows 7", version: "6.1", arch: "x86", family: "windows"
```

Рис. 2. Версия Maven

Теперь можно перейти к сборке проекта. Создайте где-нибудь папку, например, с названием **test**, перейдите в нее в консольном окне и выполните такую команду:

```
mvn archetype:generate -DgroupId=com.mycompany.app -
DartifactId=project1 -DarchetypeArtifactId=maven-
archetype-quickstart -DinteractiveMode=false
```

Самый первый запуск **Maven** может занять много времени, так как в этот момент выполняется подгрузка актуальных версий всех необходимых инструментов. Надо немного подождать. Кроме того, если вы выходите в Интернет через прокси, вы должны указать это в настройках конфигурации **Maven**. В этом случае откройте файл `{M2_HOME}/conf/settings.xml` и внесите в него информацию для прохождения прокси. Найдите в этом файле раздел **proxy**, раскомментируйте его и приведите к такому виду:

```
<proxies>
  <!-- proxy
    | Specification for one proxy, to be used in
    | connecting to the network.
    | -->
  <proxy>
    <id>optional</id>
    <active>true</active>
    <protocol>http</protocol>
    <username>proxy_user</username>
    <password>proxy_pass</password>
    <host>10.3.0.3</host>
    <port>3838</port>
    <nonProxyHosts>local.net|some.host.com</
      nonProxyHosts>
  </proxy>
</proxies>
```

Вместо **proxy_user** и **proxy_pass** надо указать пользователя и пароль для прохождения прокси, а вместо **10.3.0.3** и **3838** — адрес и номер порта прокси-сервера.

После успешного завершения этой команды вы увидите в своем консольном окне нечто такое:

```

[INFO] Parameter: basedir, Value: C:\Users\admin\Desktop\test
[INFO] Parameter: package, Value: com.mycompany.app
[INFO] Parameter: groupId, Value: com.mycompany.app
[INFO] Parameter: artifactId, Value: project1
[INFO] Parameter: packageName, Value: com.mycompany.app
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: C:\
\admin\Desktop\test\project1
-----
[INFO] BUILD SUCCESS
-----
[INFO] Total time: 59.942 s
[INFO] Finished at: 2017-06-28T14:47:20+03:00
[INFO] Final Memory: 15M/266M
-----

```

Рис. 3. Успешное создание проекта Maven

После выполнения этой команды в папке `test` будет создана папка `project1`, в соответствии со значением, указанным в командной строке в атрибуте `DartifactId`. В этой папке и будет находиться создаваемый проект. Вы увидите там две папки с именами `src` и `target`, в которых будут располагаться файлы с исходными кодами для проекта и файлы с `unit` тестами, соответственно. Создание проекта будет выполняться в соответствии со сценарием, записанным в файле `pom.xml` и расположенном непосредственно в папке `project1`. Изначально `pom.xml` выглядит так:

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/
    XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/
    POM/4.0.0
    http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.mycompany.app</groupId>
  <artifactId>project1</artifactId>
  <packaging>jar</packaging>

```

```

<version>1.0-SNAPSHOT</version>
<name>project1</name>
<url>http://maven.apache.org</url>
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>
</project>

```

Элемент `groupId`, как правило, задает доменное имя для проекта, например `com.mycompany.app`. Элементы `artifactId` и `version` формируют полное название файла проекта, а элемент `packaging` — его расширение. В нашем случае (а мы используем значения этих элементов по умолчанию, кроме `artifactId`) файл проекта получит имя `project1-1.0-SNAPSHOT.jar`.

Заготовка главного класса созданного приложения выглядит так:

```

package com.mycompany.app;
/*
 * Hello world!
 */
public class App
{
    public static void main( String[] args )
    {
        System.out.println( "Hello World!" );
    }
}

```

Теперь для сборки проекта в консольном окне надо выполнить команду:

```
mvn clean package
```

Вообще говоря, эта команда выполняет сразу два действия: удаляет компоненты создаваемого проекта, если они уже создавались ранее (*clean*), и создает проект (*package*). Если вы создаете проект в первый раз, команду **clean** можно не указывать. В папке **target** теперь должен располагаться созданный jar файл проекта с именем **project1-1.0-SNAPSHOT.jar**. Это имя создается в результате объединения значений элементов **artifactId** и **version** из файла **pom.xml**. Понятно, что приведенные значения являются значениями по умолчанию, и их можно изменять.

Maven поддерживает два режима сборки проектов: автоматический и интерактивный. За режим сборки отвечает так называемый **archetype**. Если в консольной команде присутствует команда «**mvn archetype:generate**», то содержимое файла **pom.xml** формируется на основании данных из консольной команды. Именно это имеет место в нашем случае. При использовании интерактивного режима инициализация необходимых параметров выполняется поэтапно вручную. Мы не будем рассматривать этот режим сборки.

Обратите внимание, что после выполнения последней команды в папке **target** произошёл ещё ряд изменений, среди которых:

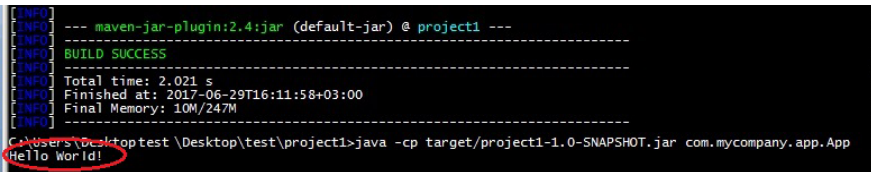
- была создана папка **surefire-reports**, в которой создан файл с отчетом **com.mycompany.app.AppTest.txt**,

- была создана папка `classes`, в которой создан скомпилированный класс приложения `App.class`,
- была создана папка `maven-archiver` с файлом свойств созданного проекта `pom.properties`.

Теперь в консольном окне надо выполнить команду:

```
java -cp target/project1-1.0-SNAPSHOT.jar
com.mycompany.app.App
```

чтобы проверить созданный проект. При активации команды вы увидите в консольном окне результат выполнения метода `main()` из созданного `Maven` главного класса приложения `App.java`.



```
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ project1 ---
[INFO] BUILD SUCCESS
[INFO] Total time: 2.021 s
[INFO] Finished at: 2017-06-29T16:11:58+03:00
[INFO] Final Memory: 10M/247M
[INFO] -----
C:\Users\Desktop\test\Desktop\test\project1>java -cp target/project1-1.0-SNAPSHOT.jar com.mycompany.app.App
Hello World!
```

Рис. 4. Успешное выполнение проекта Maven

Теперь посмотрим, как с помощью `Maven` можно добавить в созданный проект новый класс. Создайте рядом с файлом `App.java` файл с именем `Fish.java` и вставьте в него определение класса:

```
public class Fish
{
    private String name;
    private double weight;
    private double price;

    public Fish(String name, double weight, double price)
    {
```

```

        this.name=name;
        this.weight=weight;
        this.price=price;
    }

    public double getWeight()
    {
        return this.weight;
    }

    public double getPrice()
    {
        return this.price;
    }

    @Override
    public String toString()
    {
        return this.name+"  weight:"+this.weight+"
        price:"+this.price;
    }
}

```

Далее надо внести изменения в `App.java`, чтобы каким-либо образом использовать добавленный класс. Мы просто создадим объект этого класса и выведем его описание в консольное окно. Для вывода описания объекта используем переопределенный метод `toString()`:

```

package com.mycompany.app;

/**
 *
 * Hello world!
 */

```

```
public class App
{
    public static void main( String[] args )
    {
        Fish f = new Fish("salmon",2.5,180);
        System.out.println( f );
    }
}
```

Теперь нам надо собрать измененный проект. Однако его состав изменился после предыдущих сборок, ведь мы добавили новый класс. Поэтому сейчас надо указать **Maven**, что он должен выполнить компиляцию:

```
mvn clean compile
```

После успешной компиляции можно выполнять сборку проекта. В этом случае команда **clean** будет более чем уместной, поскольку перед этим мы уже выполняли сборки, и в папках остались результаты этих действий, которые лучше удалить перед новой сборкой:

```
mvn clean package
```

Теперь снова выполним наш проект:

```
java -cp target/project1-1.0-SNAPSHOT.jar
      com.mycompany.app.App
```

Вы должны получить в консольном окне описание созданного объекта класса **Fish**:

```

[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ project1 ---
[INFO] Building jar: C:\Users\admin\Desktop\test\project1\target\project1-1.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 8.654 s
[INFO] Finished at: 2017-06-30T12:07:07+03:00
[INFO] Final Memory: 22M/378M
[INFO]
C:\Users\admin\Desktop\test\project1>java -cp target/project1-1.0-SNAPSHOT.jar com.mycompany.app.App
salmon weight:2.5 price:180.0
C:\Users\admin\Desktop\test\project1>

```

Рис. 5. Успешное выполнение измененного проекта

В ОС Windows вы можете создать пакетный файл и занести в него несколько **Maven** команд, которые будут выполняться при активации такого пакетного файла. Создайте файл **project1.bat** и занесит в него такие команды:

```
call mvn clean
call mvn package
```

Теперь вам достаточно в консольном окне активировать файл **project1.bat**, и будут выполнены все **Maven** команды, указанные в этом файле.

Maven также умеет генерировать документацию по созданным проектам. Выполните в консольном окне команду

```
mvn site
```

При первой активации она будет выполняться долго, поскольку **Maven** будет обновлять требуемые зависимости. В результате выполнения команды в папке **target** будет создана еще одна папка с именем **site**, а в ней будет размещаться сайт, содержащий описание проекта. Посмотреть этот сайт вы можете, активировав файл **index**.

html. Например, в моем случае одна из страниц созданного сайта выглядит так:

project1
Last Published: 2017-07-07 | Version: 1.0-SNAPSHOT

Project Information

This document provides an overview of the various documents and links that are part of this project's general information. All of this content is automatically generated by Maven on behalf of the project.

Overview

Document	Description
Dependencies	This document lists the project's dependencies and provides information on each dependency.
Dependency Convergence	This document presents the convergence of dependency versions across the entire project, and its sub modules.
Dependency Information	This document describes how to include this project as a dependency using various dependency management tools.
About	There is currently no description associated with this project.
Plugin Management	This document lists the plugins that are defined through pluginManagement.
Plugins	This document lists the build plugins and the report plugins used by this project.
Summary	This document lists other related information of this project

Рис. 6. Сайт проекта

Этот сайт создается на основе **pom.xml**, и сейчас он не содержит никакого «персонального» описания проекта, помимо общей технической реализации. Чтобы добавить на сайт больше информации о проекте, надо внести изменения в **pom.xml**. Приведите этот файл к такому виду:

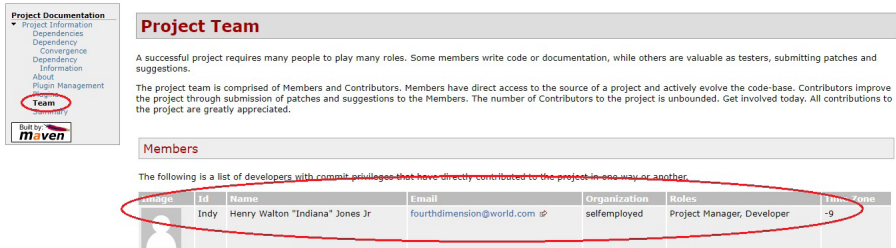
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.mycompany.app</groupId>
  <artifactId>project1</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>project1</name>
  <url>http://maven.apache.org</url>
  <description>
    This project helps me to understand the basis
    of Maven leverage
  </description>
```

```

<developers>
  <developer>
    <id>Indy</id>
    <name>Henry Walton "Indiana" Jones Jr</name>
    <email>fourthdimension@world.com</email>
    <roles>
      <role>Project Manager</role>
      <role>Developer</role>
    </roles>
    <organization>selfemployed</organization>
    <timezone>-9</timezone>
  </developer>
</developers>
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.8.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
</project>

```

В отличие от предыдущей версии этого файла, вы видите два новых добавленных элемента: **description** и **developers**. Хотя эти элементы вставлены после элемента **url**, это не значит, что они должны располагаться именно здесь. Все элементы можно добавлять в любом месте файла. Содержимое элемента **description** отображается на главной странице сайта. В этот элемент можете вставить подробное описание проекта. При наличии элемента **developers** в главном меню сайта появляется пункт **Team**, в котором отображается информация о разработчиках проекта.



Project Team

A successful project requires many people to play many roles. Some members write code or documentation, while others are valuable as testers, submitting patches and suggestions.

The project team is comprised of Members and Contributors. Members have direct access to the source of a project and actively evolve the code-base. Contributors improve the project through submission of patches and suggestions to the Members. The number of Contributors to the project is unbounded. Get Involved today. All contributions to the project are greatly appreciated.

Members

The following is a list of developers with commit privileges that have directly contributed to the project in one way or another.

Avatar	ID	Name	E-mail	Organization	Roles	Time Zone
	Indy	Henry Walton "Indiana" Jones Jr	fourthdimension@world.com	selfemployed	Project Manager, Developer	+9

Рис. 7. Сайт проекта

Вы можете добавлять в этот сайт много различной информации о проекте. Подробнее почитать об этом можно, например, по ссылке: <http://www.javaworld.com/article/2071733/java-app-dev/get-the-most-out-of-maven-2-site-generation.html>

Tomcat

Если говорить коротко, **Tomcat** — это сервер приложений, предназначенный для развертывания и выполнения **Servlet** и других компонент веб-приложений. Это продукт фирмы Apache Software Foundation, разработчика самого популярного веб-сервера Apache. **Tomcat**, как и большинство других продуктов этого известного разработчика, является **open source** продуктом.

Посмотреть подробное описание и скачать его дистрибутив можно на сайте разработчика по адресу: <http://Tomcat.apache.org/index.html>.

В этом уроке будет описана установка версии Tomcat 8, дистрибутив которой можно скачать со страницы: <http://Tomcat.apache.org/download-80.cgi>.

Выберите требуемую вам версию с учетом разрядности вашей операционной системы и скачайте на свой

диск. Для операционной системы Windows это будет zip архив. Разверните архив в какую-нибудь папку.

Перед продолжением работы обратите внимание на то, что существует известная проблема совместимости Netbeans 8.1 и Tomcat, которая часто приводит к невозможности запуска Tomcat в указанной версии Netbeans. В сети предлагается несколько способов ее решения. Мы предложим вам лучшее — просто обновите Netbeans до версии 8.2, и проблемы совместимости больше не будет. Поверьте, любой способ устранения этой проблемы для Netbeans 8.1 намного более трудоемкий, чем штатное обновление Netbeans до версии 8.2. При таком обновлении все ваши настройки и предыдущие проекты не пострадают.

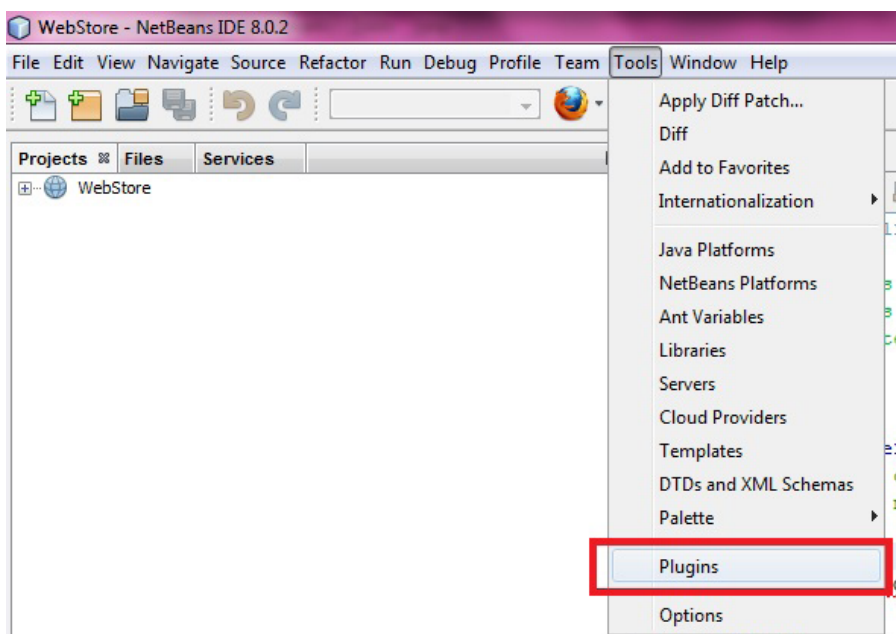


Рис. 8. Добавление плагина

К этому моменту у вас уже установлена среда разработки NetBeans (желательно, версии 8.2), но, наверное, сейчас она настроена на использование платформы Java SE, а Java EE пока является недоступной. Как узнать о том, активирована ли платформа Java EE? Проверьте, есть ли у вас в списке создаваемых проектов категории проектов Java Web и Java EE. Если их нет, значит, вам надо установить платформу Java EE. Сделать это несложно. Сначала активируйте меню **Tools – Plugins**, как показано на рисунке 8.

В появившемся окне перейдите на вкладку **Available plugins** и в окне поиска, в правом верхнем углу, введите то, что надо найти, в нашем случае — “web” или “Java EE base”. Затем выделите в левой панели компоненты «Java EE base» и «EJB and EAR», как показано на рисунке, и, наконец, нажмите кнопку **Install** (рис. 9).

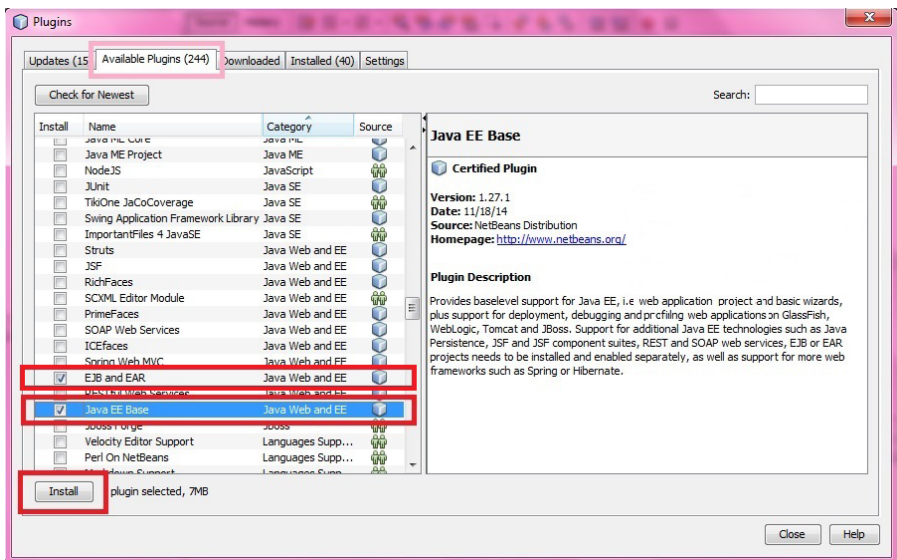


Рис. 9. Выбор и установка платформы Java EE

После установки плагина, возможно, понадобится перезагрузка NetBeans. Чтобы убедиться в том, что платформа Java EE установлена, снова перейдите в меню **File–New Project** и проверьте, появился ли там тип проектов Java Web.

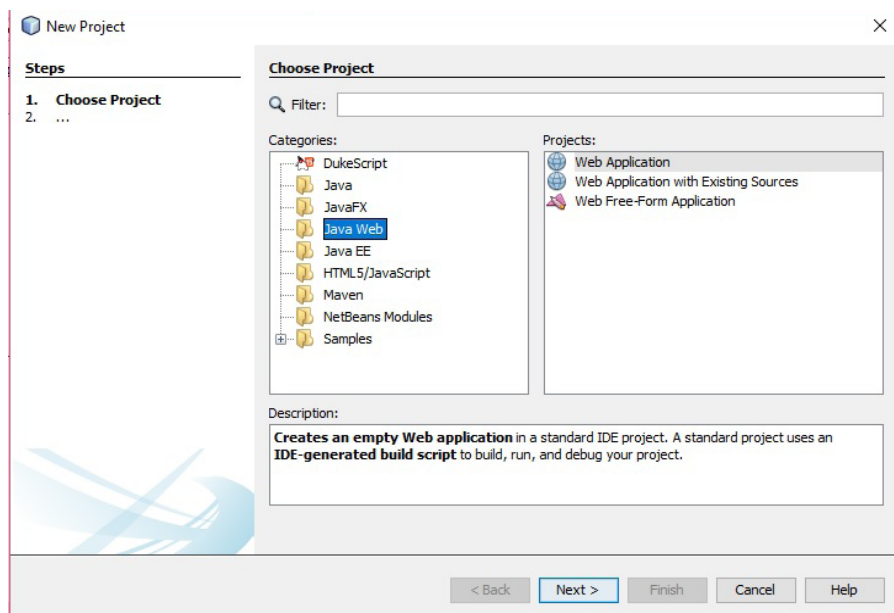


Рис. 10. Новый тип проектов Java Web

Обратите внимание, пока что мы с вами лишь установили платформу Java EE. Эта платформа позволяет создавать веб-приложения. Но любое веб-приложение требует наличия сервера приложений. Такого, как Tomcat, который ожидает нас в папке, где мы его развернули. Сейчас мы рассмотрим процесс установки самого Tomcat.

Снова активируйте меню **Tools — Servers**. Вы увидите окно, в котором отображены уже установленные

в вашем NetBeans серверы приложений. Возможно, это окно будет еще пустым, возможно, там уже будет отображен сервер **GlassFish** — это не имеет значения для того, что мы делаем. Поэтому просто нажмите в левом нижнем углу окна кнопку **Add Server**, как показано на рисунке 11.

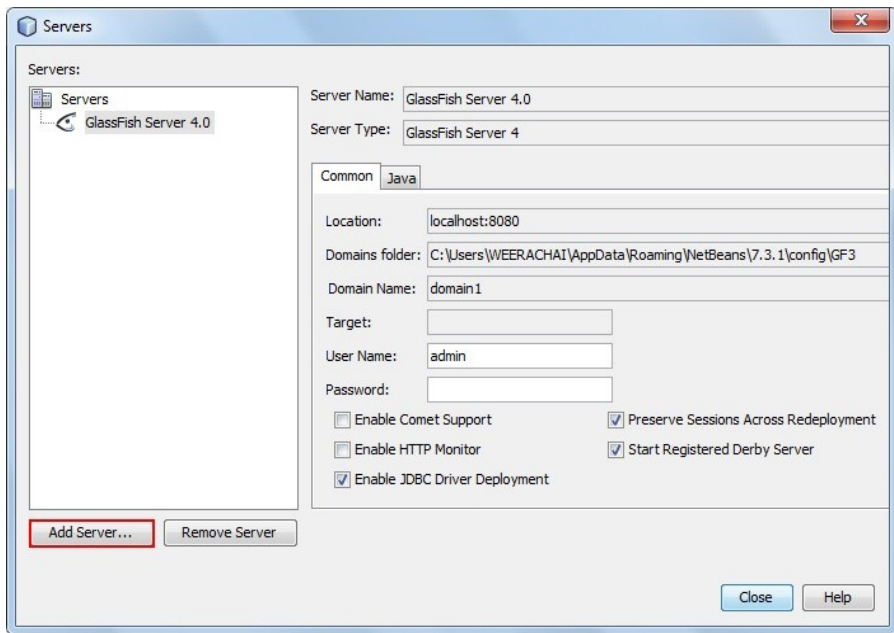


Рис. 11. Добавление сервера приложений

Теперь вы увидите новое окно, в котором сможете установить требуемый Tomcat. Сначала выберите опцию **Apache Tomcat** и нажмите кнопку **Next** (Далее) (Рис. 12).

В следующем окне укажите путь к папке, в которой вы развернули дистрибутив Tomcat и другие требуемые данные, и нажмите кнопку **Finish** (Готово) (Рис. 13).

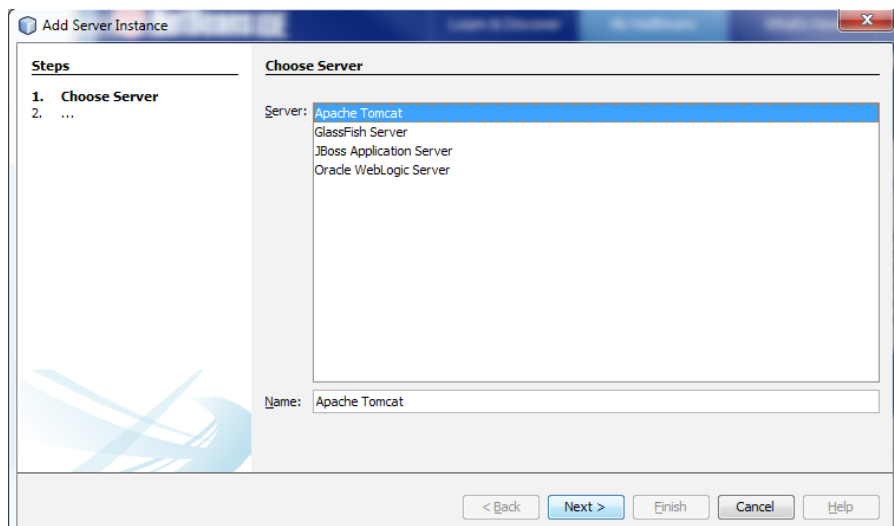


Рис. 12. Выбор Tomcat

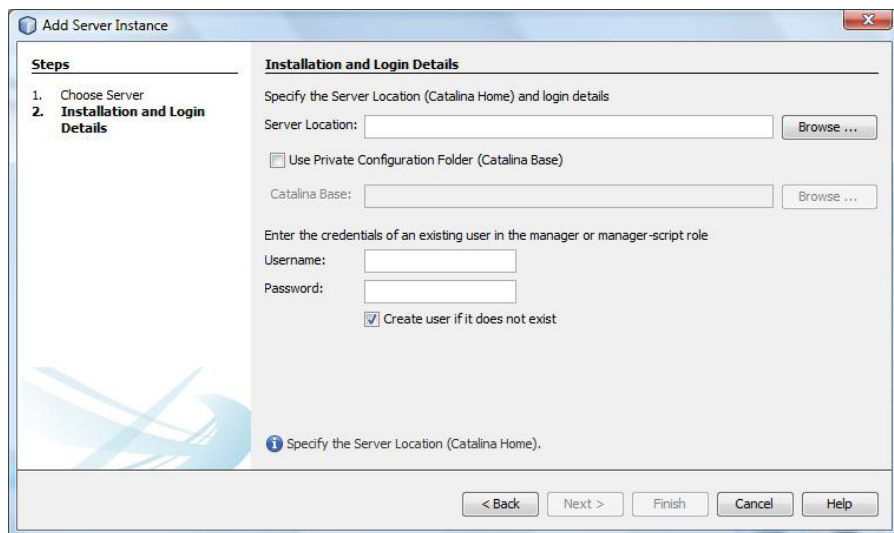


Рис. 13. Настройка Tomcat

После успешного завершения этих действий в вашем NetBeans будет установлен сервер приложений **Tomcat**,

который будет использоваться при разработке веб-приложений. Чтобы проверить установку Tomcat, перейдите в окне инспектора на вкладку **Services**, разверните узел **Servers**, выделите Tomcat и нажмите правую кнопку мышки. Из появившегося контекстного меню выберите и активируйте команду **Start**.

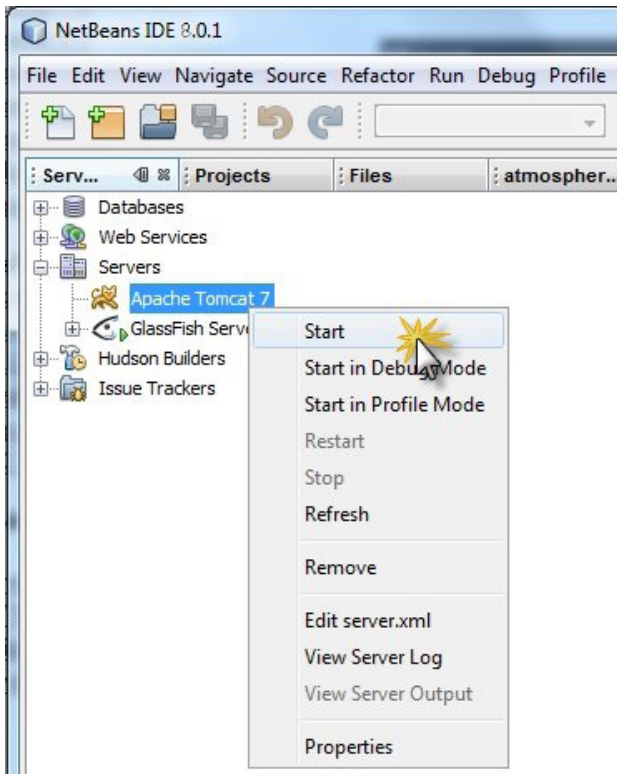


Рис. 14. Запуск Tomcat

Теперь Tomcat у вас установлен и готов к использованию. Очень скоро мы начнем с ним работать, а пока рассмотрим еще некоторые важные компоненты Java EE платформы.

JBoss

JBoss — еще один популярный сервер приложений для платформы Java EE. Этот продукт представляет собой отличную платформу для выполнения средних и больших распределенных Java EE приложений. JBoss включает в себя набор уже сконфигурированных служб, необходимых для их обслуживания. Использование JBoss обеспечивает для распределенного приложения такие опции:

- **кластеризацию** (*Clustering*) — объединение группы связанных компьютеров вместе настолько плотно, что во многих отношениях такие компьютеры работают как один;
- **балансировку загрузки приложения** (*Load Balancing*) — оптимизацию при распределении входящих запросов;
- **кеширование часто используемых данных** (*Caching*) — сохранение таких данных во временном хранилище для организации более быстрого доступа к ним;

Enterprise Java Beans — допускает использование этих компонентов, получивших широкую популярность у разработчиков Java EE.

Кроме перечисленных опций **JBoss** предоставляет еще ряд менее значимых. Понятно, что, будучи сервером приложений, **JBoss** является контейнером для развертывания и выполнения таких Java EE компонент, как **Servlet** и **JSP**.

Подробнее почитать об этом продукте вы можете на официальной странице разработчика <http://www.jboss.org/>.

Spring

Вы уже знаете что такое фреймворк. Это программный инструмент, автоматизирующий и формализующий процесс разработки приложений. Есть много разных фреймворков для разных языков программирования. Для работы на Java существует фреймворк **Spring**, который очень часто называют фреймворком фреймворков. Этим подчеркивается его значимость и глобальность для разработки на Java.

Хотя **Spring** предназначен для разработки Java EE приложений, отдельные его модули могут использоваться и в приложениях других типов. Вот основные черты этого фреймворка.

- **Spring** имеет модульную структуру, и все его модули практически независимы друг от друга. Все что вам надо знать при использовании **Spring** — это то, какой модуль вам необходим, и как его использовать. При этом вы можете совсем ничего не знать о других модулях, и это не мешает вам успешно работать со **Spring**, потому что все его модули работают независимо.
- **Spring** имеет собственный MVC модуль, который позволяет вам обходиться без других MVC-фреймворков.
- **Spring** содержит собственный API для работы с JDBC, что избавляет разработчика от необходимости выполнять огромную часть рутинной работы при использовании источников данных.
- **Spring** имеет API для обработки возникающих исключений.
- **Spring** управляет созданием объектов классов, входящих в состав вашего приложения.

- **Spring** содержит контейнеры для управления жизненным циклом объектов вашего приложения.
- Работа со **Spring** выполняется декларативно.
- Приложения, созданные с помощью **Spring**, имеют размер всего около 2МВ, что также является достоинством этого фреймворка.

При создании приложения вы должны стремиться, чтобы ваши классы были максимально независимыми друг от друга. Это позволит вам использовать их в других приложениях, а также облегчит выполнение **unit** тестирования. Но, с другой стороны, классы в приложении должны взаимодействовать друг с другом. Каким образом можно примирить эти два взаимоисключающих требования? Для этой цели **Spring** предлагает использовать паттерн проектирования *Dependency Injection (DI)*, являющийся разновидностью концепции *Inversion of Control (IoC)*. Контейнеры **IoC** являются центральной частью **Spring**.

Эти контейнеры управляют всеми аспектами жизненного цикла объектов: их созданием, конфигурацией, установлением связей между ними. Контейнеры выполняют все свои действия на основании сценария, записанного либо в XML формате, либо с помощью Java аннотаций, либо же в Java коде.

В **Spring** существуют два вида контейнеров: **BeanFactory** и **ApplicationContext**. Контейнер **BeanFactory** является более простым и предлагает основные действия, необходимые для выполнения DI. Его рекомендуется использовать в самых простых приложениях. Контейнер **ApplicationContext** включает в себя все возможности **BeanFactory** и, кроме

этого, еще может применяться в Java EE приложениях. Его надо использовать, если вы создаете веб-приложение.

Вы уже знакомы с понятием **Java bean**. Это набор формальных требований к объявлению **Java** класса, выполнение которых делает такой класс «правильным» с точки зрения **Java**, а именно — превращает класс в компоненту, которую можно использовать многократно в различных библиотеках и API. Использование **Java bean** повышает эффективность использования класса. Если вы забыли набор формальных требований к **Java bean**, повторим их еще раз. Чтобы класс стал **Java bean**, он должен:

- иметь **public** конструктор без параметров;
- быть сериализуемым;
- содержать сеттеры и геттеры для своих свойств.

Иногда еще требуется, чтобы класс содержал методы **equals()**, **hashCode()** и **toString()**.

Почему мы сейчас вспомнили о **Java bean**? Потому, что **Spring** требует, чтобы все классы приложения были **Java bean**. В этом случае контейнер **IoC** берет на себя полное управление жизненным циклом объектов вашего приложения. Вы должны специальным образом описать свои требования к процессу создания и использования объектов каждого класса, и тогда **Spring**, основываясь на этих метаданных, будет контролировать ваши объекты. Как мы упоминали выше, метаданные описываются либо в XML формате, либо с помощью **Java** аннотаций, либо же в **Java** коде.

Теперь сведем всю эту информацию вместе, для чего рассмотрим пример создания **Java** приложения

с использованием **Spring**. В этом приложении основной упор будет сделан на использование контейнеров **IoC**, чтобы продемонстрировать, как **Spring** работает с классами. Сначала создадим приложение, состоящее из одного единственного класса **Student**, создадим несколько объектов этого класса и выведем их описание. Затем добавим в приложение еще один класс — **Group**, создадим две группы студентов и выведем их описание. Главная цель этого приложения — показать, как **Spring** работает с классами.

Первое Spring приложение

Создайте в NetBeans новый проект с именем, например, **Spring1**. Когда проект будет создан, выделите его в окне инспектора и активируйте команду Свойства. В правой части появившегося окна выделите опцию **Libraries**, а затем нажмите справа кнопку «Add Library ...».

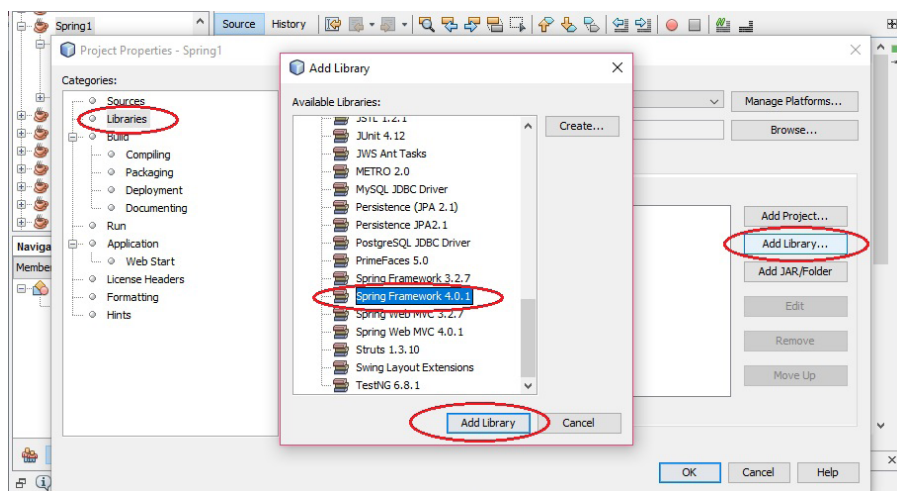


Рис. 15. Добавление Spring

В появившемся окне выберите **Spring Framework** последней версии (Рис. 15).

В нашем первом **Spring** приложении мы будем описывать свои классы в специальном конфигурационном XML файле. Он должен располагаться в папке проекта **src**, поэтому для его добавления выделите узел **SourcePackages**, активируйте контекстное меню и кликните по опции **New — Other** (рис. 16).

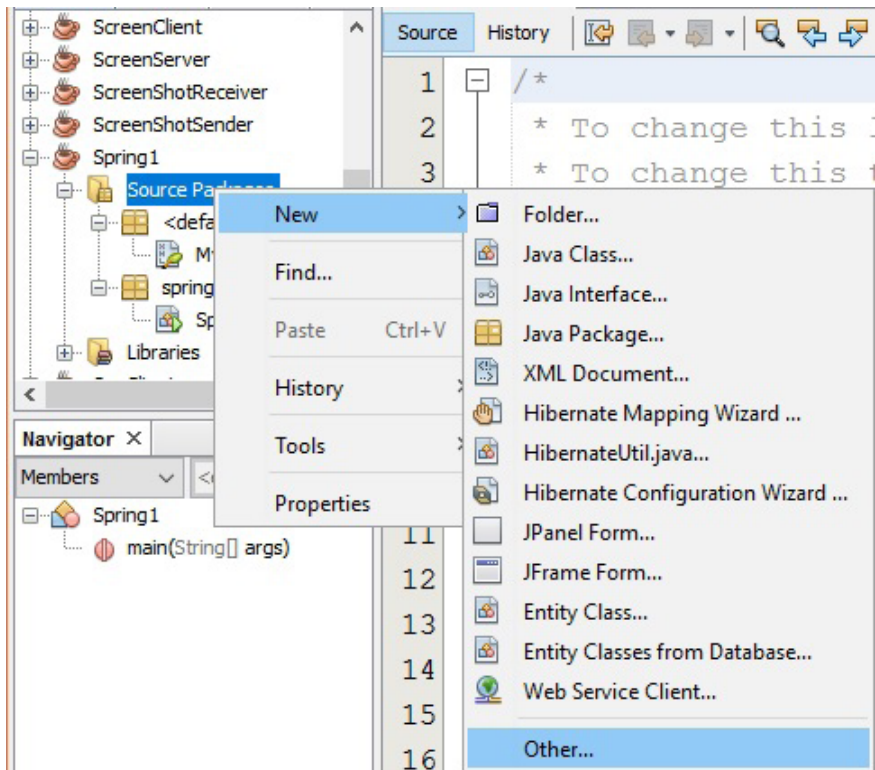


Рис. 16. Добавление конфигурационного файла Spring

В появившемся окне выберите **Other — SpringXML-Config(->)**:

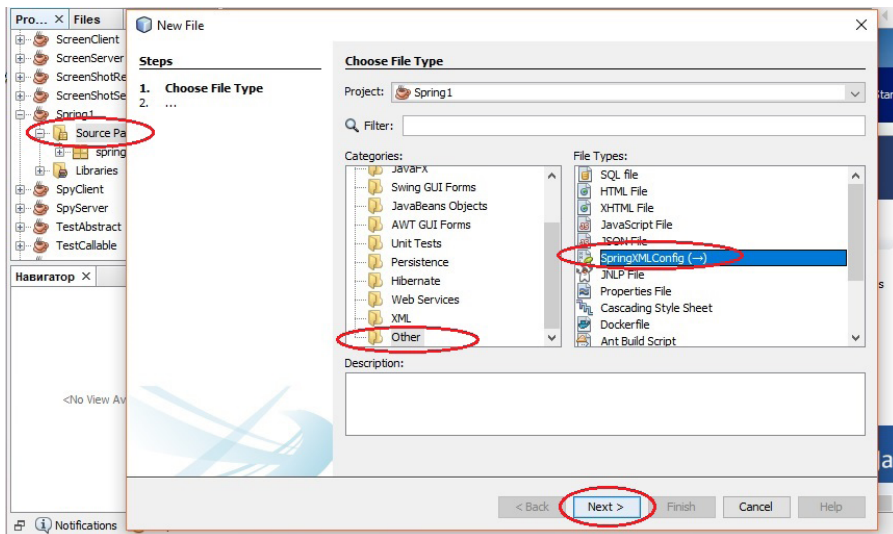


Рис. 17. Добавление конфигурационного файла Spring

Выберите для добавляемого конфигурационного файла имя и нажмите кнопку **Finish**, не отмечая никакие другие опции. Выделите добавленный файл — и увидите заготовку пустого XML файла с корневым элементом beans:

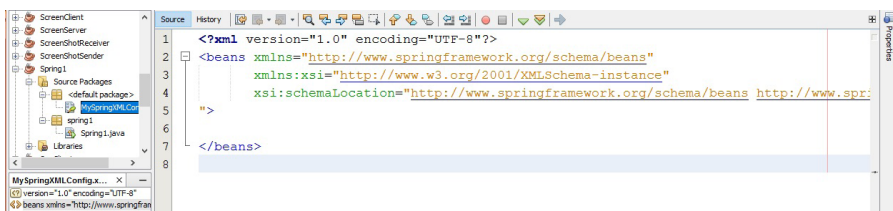


Рис. 18. Конфигурационный файл Spring

В этом файле надо описывать классы, из которых мы хотим создать приложение. Приведем сначала код класса **Student**, а затем опишем этот класс в конфигурационном файле. Создайте в проекте пакет с именем **myclass** и добавьте в него такой класс:

```
public class Student implements Serializable
{
    private String name;
    private double rate;

    public Student() {
    }

    @Override
    public String toString() {
        return "Student{" + "name=" + name + ",
            rate=" + rate + '}';
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getRate() {
        return rate;
    }

    public void setRate(double rate) {
        this.rate = rate;
    }
}
```

Обратите внимание, что класс оформлен как Java bean — это обязательное условие в случае использования **Spring**. Класс **Student** очень простой, он содержит два поля и все необходимое для их обслуживания. Теперь давайте посмотрим, как этот класс надо описывать в конфигурационном файле.

Каждый класс приложения должен описываться в конфигурационном файле парным элементом **bean**. В этом элементе необходимо указать атрибут **id** и присвоить ему уникальное значение. Обычно это имя класса, но с маленькой буквы:

```
<bean id="student">
</bean>
```

Затем с помощью атрибута **class** надо указать, какой именно класс соответствует этому **bean**. При указании имени класса надо обязательно указывать имя пакета, в котором определен класс:

```
<bean id="student" class="myclass.Student">
</bean>
```

Далее надо указать атрибут **scope**, который определяет, как будет вести себя **Spring** всякий раз, когда вы будете создавать **bean** с именем **student**. У атрибута **scope** есть пять значений, три последние из которых используются только в веб-приложениях:

<code>scope="singleton"</code>	будет создан только один объект такого bean , и при каждой попытке создать новый объект будет возвращаться этот объект;
<code>scope="prototype"</code>	при каждой попытке создать новый объект будет создаваться новый объект;
<code>scope="request"</code>	указывает, что bean существует только в пределах запроса;

<code>scope="session"</code>	указывает, что bean существует в пределах сессии;
<code>scope="global-session"</code>	указывает, что bean существует в пределах глобальной сессии.

Мы планируем создавать много объектов класса `Student`, поэтому добавляем атрибут `scope` со значением `prototype`:

```
<bean id="student" class="myclass.Student"
      scope="prototype">
</bean>
```

В элементе `bean` можно описывать поля класса и присваивать им значения по умолчанию. Полное описание нашего класса может выглядеть так:

```
<bean id="student" class="myclass.Student"
      scope="prototype">
  <property name="name" value="No Name"></property>
  <property name="rate" value="0"></property>
</bean>
```

В этом описании вам все должно быть понятно. Добавьте его в наш конфигурационный файл.

Теперь перейдем к методу `main()` в главном классе нашего приложения и добавим туда такой код:

```
public static void main(String[] args) {
    ApplicationContext context = new
        ClassPathXmlApplicationContext(
            "MySpringXMLConfig.xml");
    Student s1 = (Student) context.getBean("student");
    s1.setName("Robert");
}
```

```
s1.setRate(145.5);
System.out.println(s1);

Student s2 = context.getBean(Student.class);
System.out.println(s2);
}
```

Сначала создается контекст приложения — на основании конфигурационного файла, в котором описаны все его классы. В нашем приложении там пока описан только один класс **Student**. В дальнейшем для создания объекта какого-нибудь класса надо будет обращаться к этому контексту и вызывать метод **getBean()**, передавая ему в качестве параметра, строковый идентификатор (значение атрибута **id**) **bean**, объект которого надо создать:

```
Student s1 = (Student) context.getBean("student");
```

Запустите наше приложение, и увидите в консольном окне такой вывод:

```
Student{name=Robert, rate=145.5}
Student{name=No Name, rate=0.0}
```

Второе Spring приложение

Теперь доработаем наше приложение. Можете создать копию нашего проекта **Spring1**, назвать созданный проект **Spring2** и продолжать работать с новым проектом. А можете вносить изменения в проект **Spring1**. Добавим в проект еще один класс и посмотрим, как описывать

в конфигурационном файле несколько классов и отношения между ними.

В пакет `myclass`, в котором уже находится класс `Student`, добавим класс `Group`:

```
public class Group implements Serializable
{
    private String name;
    private List<Student> students;

    public Group()
    {
        this.students=new ArrayList<>();
        this.name="noname";
    }

    public void addStudent(Student s)
    {
        this.students.add(s);
    }

    public void removeStudent(Student s)
    {
        this.students.remove(s);
    }

    public int getGroupSize()
    {
        return this.students.size();
    }

    public String getName()
    {
        return name;
    }
}
```

```

public void setName(String name)
{
    this.name = name;
}

public List<Student> getStudents()
{
    return students;
}

public void setStudents(List<Student> students)
{
    this.students = students;
}

@Override
public String toString()
{
    String str="";
    str+="Group:\t"+name+"\n\n";

    for (Student s:students)
    {
        str+=s+"\n";
    }
    str+="\n\n";
    return str;
}
}

```

Обратите внимание, что класс **Group** тоже оформлен как Java bean, и в нем присутствует поле типа **List<Student>**. Теперь нам надо добавить в конфигурационный XML файл описание нового класса. Посмотрите, как описаны свойства класса **Group**:

```

<bean id="student" class="myclass.Student"
      scope="prototype">
  <property name="name" value="No Name"></property>
  <property name="rate" value="0"></property>
</bean>

<bean id="group" class="myclass.Group" scope="prototype">
  <property name="students" >
    <list></list>
  </property>
  <property name="name" value="No Name"></property>
</bean>

```

И, наконец, приведем код метода `main()` к такому виду:

```

ApplicationContext context = new
    ClassPathXmlApplicationContext(
        "MySpringXMLConfig.xml");
Group g=(Group) context.getBean("group");
g.setName("SP2824");
Student s=null;
Random r=new Random();
String j;

for (int i = 0; i < 10; i++)
{
    s=(Student) context.getBean("student");
    j=String.valueOf((i+1));
    s.setName("Student "+j);
    s.setRate(r.nextInt(100));
    g.addStudent(s);
}
System.out.println(g);

```

Мы создаем группу с именем "SP2824", добавляем в нее случайным образом десять студентов и выводим

описание созданной группы в консольное окно. У меня вывод приложения получился таким:

```
Group: SP2824

Student{name=Student1, rate=4.0}
Student{name=Student2, rate=67.0}
Student{name=Student3, rate=24.0}
Student{name=Student4, rate=47.0}
Student{name=Student5, rate=58.0}
Student{name=Student6, rate=51.0}
Student{name=Student7, rate=53.0}
Student{name=Student8, rate=27.0}
Student{name=Student9, rate=98.0}
Student{name=Student10, rate=1.0}
```

Как видите, пример надо брать со студента **Student9**, а остальным надо работать гораздо упорнее.

Эти два простых примера позволяют вам понять, каким образом **Spring** работает с классами. Чтобы ознакомиться с другими аспектами использования **Spring** советуем вам обратиться к сайту разработчика: <https://spring.io/guides>

Hibernate

Hibernate — очень полезный инструмент для работы с базами данных. Однако это не только средство доступа к базам данных и инструмент для выполнения запросов. **Hibernate** также является ORM системой. Термин ORM (*Object Relational Mapping*) можно перевести как «отображение объектов в связанные таблицы». Что это значит? Давайте сделаем небольшое отступление, чтобы чуть подробнее поговорить об ORM.

Сегодня большинство приложений создается в ООП стиле. При этом сущности, с которыми работает приложение, описываются классами. Вместе с тем, для хранения данных приложения чаще всего используются базы данных. В подавляющем большинстве случаев это реляционные базы данных, хранящие информацию в связанных таблицах. И вот здесь возникают некоторые проблемы, обусловленные тем, что на момент создания реляционных баз данных концепции ООП еще не существовало. Поэтому такие базы никак не поддерживают принципы ООП, чего очень хотелось бы разработчикам приложений. Для устранения этой проблемы и появились ORM системы.

Главный принцип работы таких систем заключается в том, что они создают однозначное соответствие между классами приложения и таблицами в базах данных. Упрощенно такое соответствие можно рассматривать так. Для каждого класса приложения создается таблица в БД. Поля в этой таблице по именам и типам данных соответствуют свойствам класса. Каждый объект такого класса может быть превращен в одну строку в соответствующей таблице. А любую строку такой таблицы можно превратить в объект соответствующего класса. Вообще говоря, такое объяснение довольно грубое. Дело в том, что чаще всего один класс представляется в БД не одной таблицей, а группой связанных таблиц. Да и сущность, с которой работает приложение, может описываться не одним классом, а несколькими, о которых говорят как об «объектной модели». Поэтому надо говорить не о соответствии «класс — таблица», а, скорее, о соответствии «объектная модель — группа связанных таблиц».

Есть и другие проблемы, которые решаются с помощью ORM систем. Например, как быть с наследованием? Как отображать в БД классы, связанные наследованием? Как определять «одинаковость» объектов? В БД идентичность определяется совпадением **primary key**, а в ООП есть понятие одинаковых объектов (**a==b**) и совпадающих объектов (**a.equals(b)**). Как устанавливать соответствие между типами данных СУБД и Java? Если вы задумаетесь, то согласитесь, что подобных проблем при таком соответствии возникает много.

Так вот, **Hibernate** и есть ORM система, решающая все такие проблемы и включающая в себя также средства доступа и работы с БД. Основные характеристики **Hibernate** таковы:

- это **open source** система;
- это очень быстрая система, во многом благодаря эффективному использованию кеширования;
- она включает в себя язык HQL (*Hibernate Query Language*) — ООП версию языка SQL, который позволяет писать запросы к БД, независимые от конкретного сервера БД;
- позволяет автоматически создавать таблицы в БД;
- упрощает работу с многотабличными запросами;
- позволяет работать практически со всеми известными серверами БД;

Функционально в составе **Hibernate** можно выделить такие компоненты, как API для выполнения CRUD операций над объектами классов, API для конфигурирования соответствия между классами и таблицами и некоторые другие.

Загрузить Hibernate можно со страницы разработчика <http://hibernate.org/orm/downloads/>.

Рассмотреть способы использования Hibernate можно по адресу <https://netbeans.org/kb/docs/web/hibernate-webapp.html>.

Далее в этом уроке, после того, как мы познакомимся с использованием баз данных, мы создадим приложение с Hibernate.

Понятие сервлета

Сервлет представляет собой приложение, выполняющееся на веб сервере или, гораздо чаще, на сервере приложений (Tomcat, GlassFish и др.). Сервлет располагается между клиентской и серверной частью приложения и предназначен для обработки клиентских запросов. Обработывая клиентские запросы, сервлет может обращаться к БД, к веб службам или же формировать **Response** самостоятельно.

По своей природе сервлеты являются Java классами, наследующими класс **HttpServlet**. Реализация этих классов базируется на пакетах **javax.servlet** и **javax.servlet.http**, входящими в состав Java EE. В классе сервлета определены методы, управляющие его жизненным циклом. Это такие методы, как:

- **init()** — вызывается единожды и инициализирует сервлет. Сервлет обычно создается, когда клиент обращается к **Url**, соответствующему сервлету. Однако можно сделать и так, чтобы сервлет инициализировался сразу при запуске сервера;
- **service()** — вызывается всякий раз, когда сервлет получает клиентский запрос. Это основной метод сервлета. В нем выполняется работа, для которой сервлет создан. Каждый раз при получении нового запроса от клиента сервер вызывает в новом потоке метод **service()**, далее уже сам этот метод проверяет, по какому HTTP методу пришел запрос (**GET**, **POST**,

DELETE, PUT), и вызывает соответствующий метод `doGet()`, `doPost()`, `doDelete()` и т.п. Обратите внимание, что метод `service()` вызывается контейнером, в котором размещен сервлет, и программисту не надо самостоятельно вызывать его. Задача программиста — переопределить методы `doGet()`, `doPost()`, `doDelete()` и т.п.;

- `destroy()` — вызывается единственный раз и завершает работу сервлета, после чего сервлет удаляется сборщиком мусора. В этом методе надо выполнять такие действия, как отключение от серверов БД, завершение потоков-демонов, сохранение куки файлов и т.п.

После этой начальной информации рассмотрим примеры использования сервлетов. Для работы с ними надо

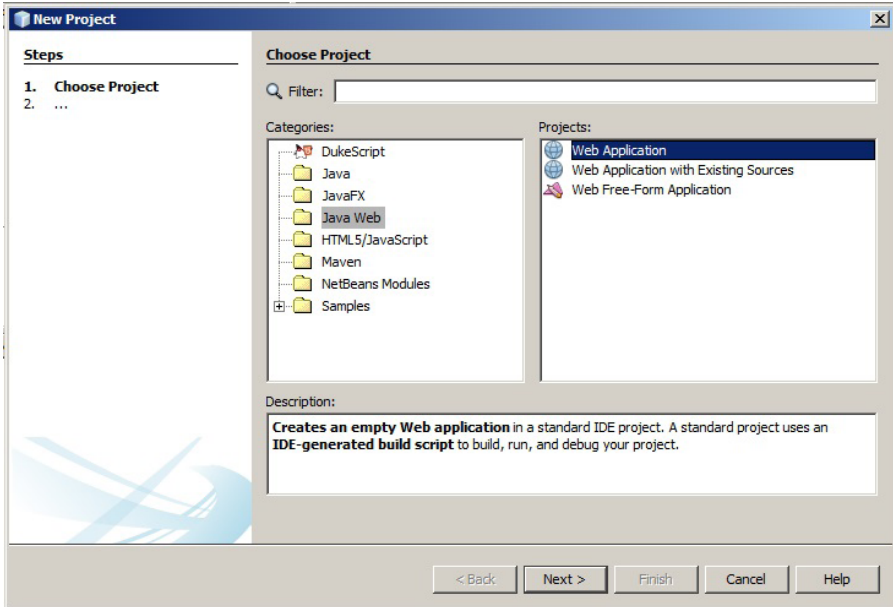


Рис. 19. Создание веб приложения

иметь установленной платформу Java EE и какой-нибудь сервер приложений. У нас к этому моменту оба условия выполнены: установлены Java EE и Tomcat. Поэтому перейдем к созданию нового приложения.

Запустите NetBeans и создайте новый проект по шаблону Java Web (см. рис. 19).

Назовите проект, например **MyServlet1**, и перейдите к следующему окну, где вам предложат выбрать сервер для хостинга создаваемого приложения. Выберите опцию Apache Tomcat:

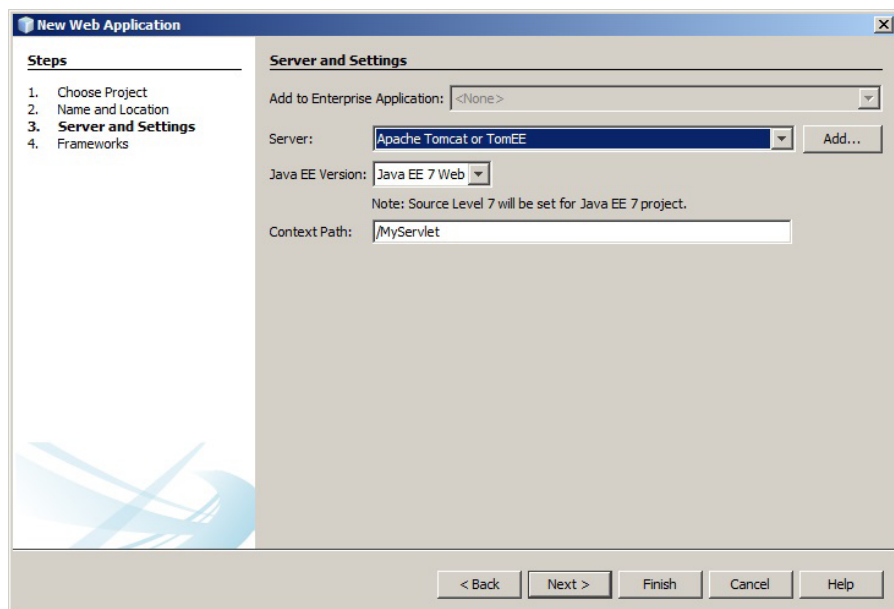


Рис. 20. Выбор сервера

В корневой папке созданного приложения вы увидите файл **index.html**, это страница входа созданного приложения. Приведите разметку элемента **body** этого файла к такому виду:

нажмите правую кнопку мышки, затем — **New**, затем — **Servlet** (рис. 21).

В качестве имени сервлета укажите **GreetingServlet**. Вы увидите шаблон класса сервлета, производного от **HttpServlet**. В этом классе нас будет интересовать метод **processRequest()** с двумя параметрами — **HttpServletRequest request** и **HttpServletResponse response**. Первый параметр содержит входящий запрос от клиента. Во второй надо записывать сформированный **response** на полученный **request**. Приведите этот метод к такому виду:

```
protected void processRequest(HttpServletRequest request,
    HttpServletResponse response) throws ServletException,
    IOException {
    response.setContentType("text/html;
        charset=UTF-8");
    PrintWriter out = response.getWriter();
    String login = request.getParameter("login").
        toString();
    String pass = request.getParameter("pass").
        toString();
    out.println("<html>");
    out.println("<head>");
    out.println("<title>Servlet GreetingServlet
        </title>");
    out.println("</head>");
    out.println("<body>");
    out.println("<h1>Servlet GreetingServlet at " +
        request.getContextPath () + "</h1>");
    out.println("<p>Welcome, " + login + ",
        your pass is " + pass + "</p>");
    out.println("</body>");
    out.println("</html>");
    out.close();
}
```

В этом методе выполняются очень простые действия. Сначала мы задаем требуемую кодировку и связываем с объектом `response` выходной поток, чтобы иметь возможность писать разметку в `response`, который будет получен клиентом как результат обработки формы. Затем из объекта `request` с помощью метода `getParameter()` мы по имени элементов управления получаем значения, занесенные в форму. Далее идет формирование разметки для возвращаемой клиенту страницы.

Запустите приложение, и в браузере откроется созданная форма. Занесите в нее какие-нибудь данные и нажмите кнопку `submit`. Тем самым вы создадите запрос, который будет отправлен нашему сервлету. Там его обработает метод `processRequest()` и вернет страницу ответа. У меня это выглядит так:

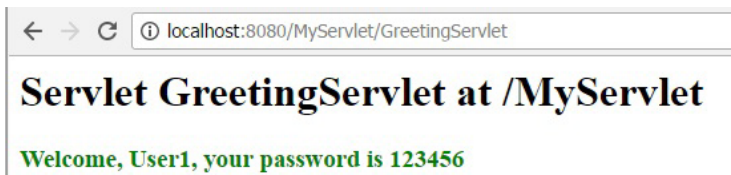


Рис. 22. Активация сервлета

Поговорим о том, что вы увидели в этом примере. Вы увидели типичное веб приложение с клиентской частью в браузере и с генерацией запроса с помощью формы. Обработчиком запроса является специфический Java класс — сервлет `GreetingServlet`. Именно сервлет получил запрос из нашей формы, сформировал для этого запроса `response` и прислал его обратно в браузер клиента. Вся логика в сервлете выполнялась в методе `processRequest()`. Возможно у вас возник вопрос: почему мы не описали этот

метод среди других методов сервлета (`init()`, `service()`, `destroy()`)? Дело в том, что метод `processRequest()` не является частью класса сервлета. Этот метод добавляется средой NetBeans и является удобным местом для выполнения обработки запросов.

Давайте выполним еще один пример для закрепления знакомства с сервлетами. В этот раз приложение будет интереснее и сложнее. Оно будет предлагать клиентам выгружать на сервер картинки. Картинки будут накапливаться в специальной папке `images`. Это все будет выполняться в одном сервлете. Затем будет создан второй сервлет, который будет предоставлять клиентам возможность выбрать какую-нибудь из загруженных картинок в выпадающем списке и просмотреть ее в браузере.

Создайте в NetBeans новое веб приложение, как мы делали это в предыдущем примере. У меня это приложение называется `ServletImages`. Откройте файл `index.html` и вставьте в него такую разметку:

```
<h2>Select files for upload</h2>
<form method="POST" action="Servlet1"
enctype="multipart/form-data" >
    <input type="file" name="file" multiple="multiple"/>

```

На главной странице приложения мы создаем две формы. Одна предназначена для выполнения множественной загрузки файлов. Об этом говорят атрибут `enctype` у формы, и атрибут `multiple` у элемента `input`. Обработчиком этой формы является сервлет `Servlet1`. Вторая предназначена для вызова второго сервлета с именем `Servlet2`. В свою очередь, `Servlet2` создает страницу, на которой отображает форму с элементом `select` со всеми картинками из папки `images`, и выполняет отображение картинки, выбранной пользователем в элементе `select`.

Теперь добавьте в приложение два сервлета с именами `Servlet1` и `Servlet2`. Рассмотрим их содержимое. Вы уже должны были обратить внимание, что перед объявлением класса сервлета автоматически вставляется аннотация `@WebServlet` с атрибутами `name` и `urlPattern`. Такая же аннотация будет и в нашем случае, но, кроме нее, надо добавить еще одну аннотацию, обеспечивающую много-файловую загрузку. Аннотации перед нашим первым сервлетом должны быть такие:

```
@WebServlet(name = "Servlet1",
            urlPatterns = {"/Servlet1"})

@MultipartConfig (
    fileSizeThreshold = 1024*1024*2,
    maxFileSize = 1024*1024*10,
    maxRequestSize = 1024*1024*50
)

public class Servlet1 extends HttpServlet
{
}
```

Теперь рассмотрим код внутри сервлета. Нам надо определить путь к папке, где мы планируем собирать картинки, поэтому в самом начале класса добавим такую строку, объявляющую константу **SAVEDIR** с путем к папке:

```
public final static String SAVEDIR="Images";
```

Такое объявление пути приведет к тому, что папка **Images** будет создана в проекте по пути **build/web/Images**.

В этот раз мы не будем выполнять обработку запросов клиента в методе **processRequest()**, как это было в предыдущем примере. Мы рассмотрим использование метода **doPost()**, что больше соответствует логике нашего приложения, поскольку форма отправляет запрос по методу POST. Однако в методе **processRequest()** мы все-таки добавим одну строку, которая будет переадресовывать клиента обратно на главную страницу приложения после выполнения метода **doPost()**. Эту строку добавим в начале метода:

```
protected void processRequest(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException
{
    response.setContentType("text/html;charset=UTF-8");
    response.sendRedirect("index.html"); //added line
    // rest of the default code
}
```

Больше в этом методе ничего не изменяем. Перейдем к рассмотрению метода **doPost()**. Поскольку наша форма пересылает запрос по HTTP методу POST, он попадет на метод **doPost()**. У вас не должно возникать путаницы с двумя словами «метод» в предыдущем предложении.

Первое вхождение слова «метод» означает «стандартизированный в протоколе HTTP способ передачи данных», а второе — «метод `doPost()` класса `Servlet1`». Добавим в метод `doPost()` такой код:

```
protected void doPost(HttpServletRequest request,
    HttpServletResponse response) throws ServletException,
    IOException
{
    if(request.getParts().size()>0)
    {
        this.doUpload(request);
    }
    processRequest(request, response);
}
```

Мы проверяем, выбрал ли клиент какие-либо файлы для выгрузки. Если да, то вызываем некий метод `doUpload()`, в котором обрабатываем выгрузку. Затем, независимо от того, была выгрузка или ее не было, мы передаем запрос на дальнейшую обработку методу `processRequest()`. `doUpload()` — это наш метод, в который вынесена обработка. Он может выглядеть так:

```
private void doUpload(HttpServletRequest request)
{
    String root=request.getServletContext().
        getRealPath("");
    String savePath=root+File.separator+Servlet1.SAVEDIR;

    File dir=new File(savePath);

    if(!dir.exists())
        dir.mkdir();
}
```

```

String fn="";
InputStream in=null;
BufferedImage img=null;
FileOutputStream fos=null;
try
{
    for(Part p : request.getParts())
    {
        fn=p.getSubmittedFileName();
        in=p.getInputStream();
        byte [] b=new byte[in.available()];
        in.read(b);
        fos = new FileOutputStream(savePath+File.
            separator+fn);
        fos.write(b);
        fos.close();
    }
} catch (IOException ex)
{
    Logger.getLogger(Servlet1.class.getName()).
        log(Level.SEVERE, null, ex);
} catch (ServletException ex)
{
    Logger.getLogger(Servlet1.class.getName()).
        log(Level.SEVERE, null, ex);
}
}

```

Действия в этом методе просты и понятны:

- конструируем путь к папке **Images**;
- создаем папку, если она еще не создана;
- готовим потоки для записи бинарного контента;
- перебираем в цикле файлы, присланные в запросе, и сохраняем их в папке **Images**.

Вот и все действия, что выполняются в первом сервлете. Перейдем к рассмотрению второго.

В этом сервлете обработка опять будет выполняться в методе `processRequest()`. Здесь будет создаваться форма с элементом `select`, заполненная информацией о наших картинках из папки `Images`, и здесь же будет обрабатываться выбор клиента. Обработка сводится к созданию элемента `img` для выбранной картинки.

Приведите метод `processRequest()` в `Servlet2` к такому виду:

```
protected void processRequest(HttpServletRequest request,
    HttpServletResponse response) throws ServletException,
    IOException
{
    response.setContentType("text/html;charset=UTF-8");
    PrintWriter out = response.getWriter();

    /* TODO output your page here. You may use
       following sample code.
    */
    out.println("<!DOCTYPE html>");
    out.println("<html>");
    out.println("<head>");
    out.println("<title>Servlet Servlet2</title>");
    out.println("</head>");
    out.println("<body>");

    //
    String btn1=request.getParameter("showImages");
    String select=request.getParameter("lImages");

    if( (btn1!=null && btn1.equals("View Page")) ||
        select!=null)
    {
```

```

String root=request.getServletContext().
    getRealPath("");
String savePath=root+File.separator+Servlet1.
    SAVEDIR;
File dir=new File(savePath);
File f=null;
out.println("<h2>Select image to view</h2>");
out.println("<form method='POST'
    action='Servlet2' >");
out.println("<select name='lImages'>");
for(String fname:dir.list())
{
    f=new File(savePath+File.separator+fname);
    if(!f.isFile())
        continue;
    out.println("<option>"+fname+"</option>");
}
out.println("</select>");
out.println("<input type='submit'
    value='Show image'>");
out.println("</form>");

if(select!=null)
{
    Path path = Paths.get(savePath+File.
        separator+select);
    byte[] data = Files.readAllBytes(path);

    StringBuilder sb = new StringBuilder();
    sb.append("data:image/png;base64,");
    sb.append(new String(Base64.getEncoder().
        encode(data)));

    out.println("<img src='"+sb.toString()+"'>");
}
}
//

```

```

out.println("<h4>The image is presented by
    Servlet2 at " + request.getContextPath() +
    "</h4>");
out.println("</body>");
out.println("</html>");

out.flush();
out.close();
}

```

Вы видите обработчик формы, в котором выполняется отображение файла, выбранного в элементе `select`. Все действия в этом коде вам понятны. Запустите это приложение, выполните загрузку нескольких картинок. Убедитесь, что папка `Images` создана и выбранные картинки находятся в ней. Выберите какой-либо графический файл для просмотра. У меня результат выглядит так:

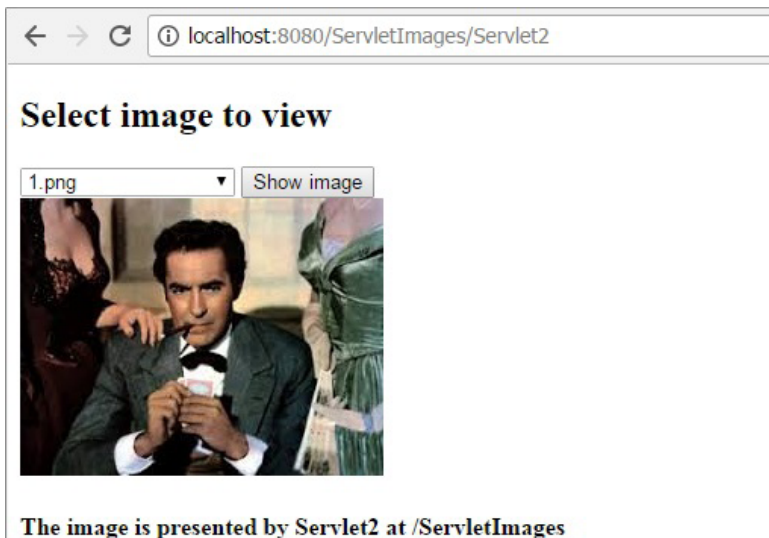


Рис. 23. Активация сервлета

Этот пример полнее раскрывает возможности сервлета. Вы видите определенную пользовательскую логику, созданную с помощью двух классов сервлетов, использование разных методов, переадресацию, работу с папками, отображение бинарного контента. У сервлетов нет никаких функциональных ограничений, они могут работать с базами данных, с веб-службами и другими компонентами. Сервлеты очень широко используются в веб-приложениях Java.

Источники данных

JDBC

Вообще говоря, источником данных не обязательно должна быть база данных. Это может быть и файл, и коллекция. Но именно базы данных практически стали стандартным источником данных. Для работы с базами данных в Java SE включен пакет `java.sql`, содержащий компоненту JDBC (*Java DataBase Connectivity*). JDBC основывается на понятии драйвера. Для подключения к конкретной БД надо динамически загрузить соответствующий драйвер и передать ему аналог строки подключения, который в Java называется `URL`. Основную роль в JDBC играют такие интерфейсы, как `Connection`, `Statement`, `PreparedStatement`, `CallableStatement` и `ResultSet`. Драйвер каждой конкретной БД реализует эти интерфейсы, как ему необходимо.

Алгоритм работы с БД выглядит таким образом:

- регистрация драйвера для конкретного сервера БД;
- выполнение соединения с сервером БД (`Connection`);
- создание и выполнение запроса к БД (`Statement` или `PreparedStatement`);
- извлечение результата выполненного запроса (`ResultSet`).

Загрузка и инициализация драйвера происходит при выполнении строки кода:

```
Class.forName(driver);
```

здесь `driver` — специальный строковый описатель драйвера.

Выполнение подключения к серверу БД выглядит так:

```
Connection connection = DriverManager.  
    getConnection(driverUrl);
```

здесь `driverUrl` — уникальная для каждого сервера БД строка специального вида.

Дальнейшие действия специфичны для конкретных выполняемых запросов.

Работа с JDBC практически закрывает от пользователя реализацию конкретной БД. Другими словами, код, использующий JDBC, очень мало зависит от специфики БД. В случае изменения БД, в Java коде надо будет делать очень мало правок, а возможно, не понадобится их делать вовсе, за исключением нескольких строк подключения драйвера.

Рассмотрим пример использования БД. Напишем приложение, которое выполнит подключение к MySQL и продемонстрирует выполнение основных действий с БД. У вас уже должен быть установлен экземпляр MySQL и `PhpMyAdmin`. Запустите `PhpMyAdmin` и создайте базу данных с именем `jtest`. Затем запустите NetBeans и создайте новый консольный проект. Мой проект будет называться `TryJDBC`.

Поскольку мы будем работать с сервером MySQL, нам надо добавить в состав приложения JDBC драйвер для работы с этим сервером. Этот драйвер можно загрузить со страницы <https://www.mysql.com/products/connector/>. Перейдите на эту страницу и активируйте выделенную ссылку:

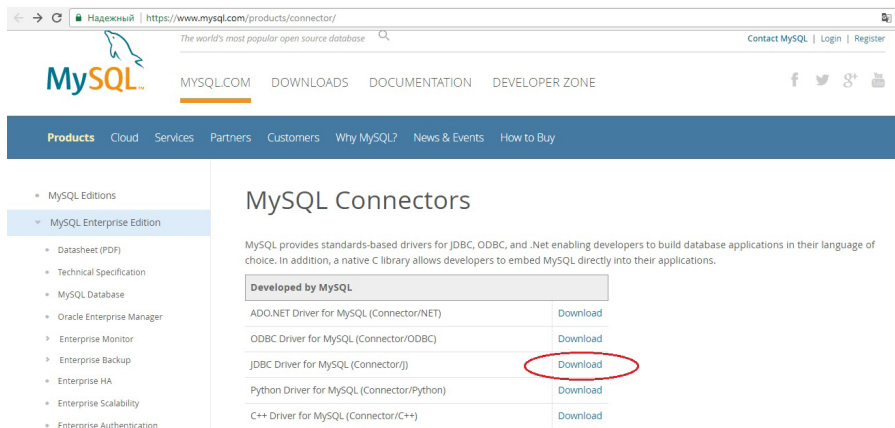


Рис. 24. Загрузка драйвера MySQL

На следующей странице выберите дистрибутив, соответствующий вашей платформе:



Рис. 25. Выбор дистрибутива драйвера MySQL

Наконец, на следующей странице загрузки нажмите внизу ссылку, позволяющую загрузить выбранный дистрибутив без регистрации (рис. 26).

Распакуйте скачанный архив в какую-либо папку на своем диске. Теперь вы можете добавить в созданный новый проект скачанный драйвер. Запомните, это надо делать для каждого проекта, в котором вы хотите работать

- MySQL SUSE Repository
- MySQL Community Server
- MySQL Cluster
- MySQL Router
- MySQL Utilities
- MySQL Shell
- MySQL Workbench
- » MySQL Connectors
- Other Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system
- Comment in the MySQL Documentation

Login »

using my Oracle Web account

Sign Up »

for an Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can signup for a free account by clicking the Sign Up link and following the instructions.

No thanks, just start my download.

Рис. 26. Завершение загрузки драйвера MySQL

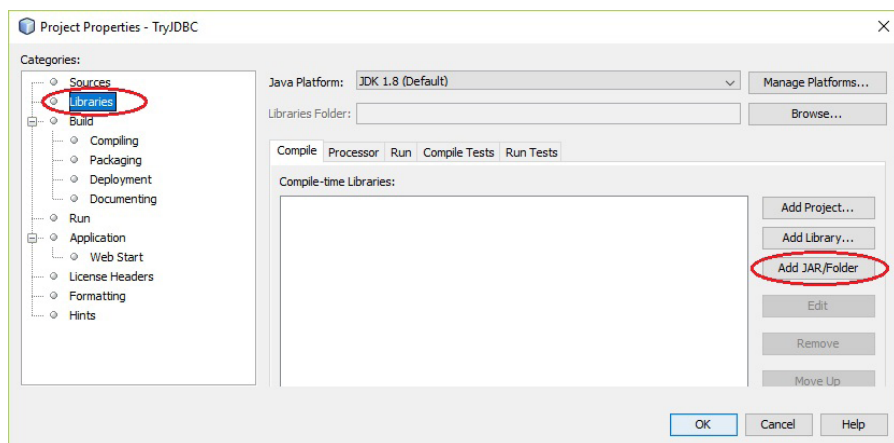


Рис. 27. Добавление драйвера JDBC MySQL в проект

с JDBC. Выделите в окне инспектора проект, активируйте его свойства и выделите в левой части окна опцию **libraries**. В правой части следующего окна нажмите кнопку **Add JAR/Folder**, перейдите в папку, куда вы распаковали скачанный драйвер, выберите файл **mysql-connector-java-5.1.42-bin.jar** и нажмите кнопку ОК. Возможно, вы

скачаете другую версию драйвера, тогда имя файла будет отличаться от приведенного (рис. 27).

Теперь мы можем перейти к написанию кода. Наше приложение выполнит подключение к созданной БД `jtest`, создаст в ней таблицу, занесет в нее несколько записей, а затем выведет данные из таблицы в консольное окно.

Вынесем работу с БД в отдельный класс. В этом классе создадим методы для выполнения соединения с БД и метод для выполнения запросов `select`. Запросы, которые не возвращают данные из БД, будем выполнять другим способом, вне этого класса. При создании объекта этого класса будем передавать конструктору данные, идентифицирующие адрес сервера, пользователя и имя конкретной БД. Добавьте в состав проекта такой класс:

```
public class DbManager {
    private String host;        //server address
    private String user;        //user name
    private String pass;        //user password
    private String dbName;      //DB name
    private Connection conn;    //connection object

    public DbManager(String host, String user,
                      String pass, String dbName)
    {
        this.conn=null;
        this.host=host;
        this.user=user;
        this.pass=pass;
        this.dbName=dbName;

        try
        {
            //driver registration
```

```

        Class.forName("com.mysql.jdbc.Driver");
    } catch (ClassNotFoundException ex)
    {
        Logger.getLogger(DbManager.class.getName()).
            log(Level.SEVERE, null, ex);
    }
}

public Connection connect()
{
    String url="jdbc:mysql:
        //" + this.host + "/" + this.dbName +
        "?useUnicode=true&characterEncoding=
        utf-8";
    try
    {
        this.conn=DriverManager.
            getConnection(url,this.user,this.pass);
        return this.conn;
    } catch (SQLException ex)
    {
        Logger.getLogger(DbManager.class.getName()).
            log(Level.SEVERE, null, ex);
        return null;
    }
}

public ResultSet getSelectQuery(String sql,
    Connection conn)
{
    Statement comm = null;
    ResultSet set = null;
    try
    {
        comm = (Statement) conn.createStatement();
        set = comm.executeQuery(sql);
    }
}

```

```

        catch (SQLException ex)
        {
            Logger.getLogger(DbManager.class.getName()).
                log(Level.SEVERE, null, ex);
        }
        return set;
    }
}

```

В конструкторе класса выполняется загрузка MySQL драйвера. Чтобы загрузить драйвер именно для MySQL, методу `Class.forName()` надо передать в качестве параметра строку с именем драйвера вида: `"com.mysql.jdbc.Driver"`. Это предопределенная строка. Для подключения к другим СУБД она будет другой. Например:

- для MS SQL Server — `"com.microsoft.jdbc.sqlserver.SQLServerDriver"`;
- для Oracle — `"oracle.jdbc.driver.OracleDriver"`;
- для PostgreSQL — `"postgresql.Driver"`.

Вы всегда можете узнать, как должно выглядеть имя JDBC драйвера на сайте разработчика требуемой вам СУБД.

Метод `connect()` выполняет подключение к указанной в строке `url` базе данных. Обратите внимание, что формат строки `url` тоже строго фиксирован и уникален для каждого сервера. Для случая MySQL формат этой строки должен быть таким:

```

"jdbc:mysql://host/dbName?useUnicode=
true&characterEncoding=utf-8"

```

где:

- **host** — адрес сервера;
- **dbName** — имя БД;

Далее в этом методе инициализируется объект **Connection**, через который потом будут выполняться запросы к БД.

Теперь перейдем в метод **main()** и будем вставлять туда фрагменты кода, объясняя их работу, где это необходимо. Добавьте две следующие строки:

```
DbManager db = new DbManager("localhost:3307",
                             "root", "123456", "jtest");
Connection conn = db.connect();
```

Создаем объект класса **DbManager**, указав ему данные, необходимые для подключения к нашей БД. Затем вызываем метод **connect()** созданного нами класса **DbManager**, чтобы получить объект типа **Connection**. Это один из центральных типов при работе с БД. Создав этот объект, мы сможем с его помощью (вызовом метода **createStatement()**) создать объект типа **Statement**. А последний позволит нам выполнять запросы к серверу БД.

```
Statement statement1 = conn.createStatement();
```

У объекта **Statement** есть метод **execute()**, который принимает строку с SQL запросом и выполняет этот запрос. Мы выполняем запрос, создающий таблицу **Student** с полями **id**, **name** и **rate**. Чтобы запрос не выполнялся при каждом запуске приложения, мы включили в него фразу *if not exists*.

```
statement1.execute("create table if not exists
    Student(" + "id integer primary
    key auto_increment, " + "name
    varchar(100), " + "rate double);");
```

На этом этапе вы можете запустить приложение, и если вы набрали код без ошибок, то приложение успешно выполнится, и в базе данных будет создана таблица **Student**. Убедитесь, что таблица создана.

Теперь рассмотрим выполнение запросов **insert**, чтобы добавить в созданную таблицу несколько записей. Это можно сделать так:

```
String s1_name="Corwin of Amber";
double s1_rate=190;
String ins1="insert into Student(name, rate)
    values('" + s1_name + "', '" + s1_rate + "')";
statement.execute(ins1);
```

Готовим переменные с требуемыми данными, вставляем эти данные в строку запроса, а затем снова выполняем запрос с помощью метода **execute()**, как и при создании таблицы. Такой способ выполнения **insert** будет работать, однако пользоваться им не стоит. Во-первых, вшивать данные в тело запроса, следить при этом за кавычками — работа рутинная. Во-вторых, данные из переменных при этом никак не проверяются, что является брешью для SQL инъекции.

Более предпочтительным является использование типа **PreparedStatement**, позволяющего создавать параметризованные запросы, в которых вместо параметров можно указывать плейсхолдеры, а затем связывать с каждым плейсхолдером требуемые данные.

Объект создается вызовом метода `prepareStatement()` от имени того же объекта `Connection`. В качестве параметра методу `prepareStatement()` надо передать строку запроса, в которой вместо каждого вводимого значения можно указать плейсхолдер «?». Затем, учитывая тип значения, вызовами методов `setBoolean()`, `setString()`, `setDouble()`, `setInt()` и другими подобными надо привязать значение для каждого плейсхолдера, указывая первым параметром номер плейсхолдера (начиная с 1).

```
String s2_name="Han Solo";
double s2_rate=180;
PreparedStatement statement2 = conn.prepareStatement
    ("insert into Student(name,rate) values(?,?)");
statement2.setString(1, s2_name); //first value is of
                                   //type String
statement2.setDouble(2, s2_rate); //second value is
                                   //of type Double
statement2.executeUpdate();
```

У вас может возникнуть вопрос, чем `PreparedStatement` отличается от `Statement`. Отличия есть и они существенны:

- `PreparedStatement` выполняются значительно быстрее, чем `Statement`, потому, что такие запросы компилируются;
- данные для `Statement` являются статическими, в то время как для `PreparedStatement` их можно добавлять динамически;
- `PreparedStatement` автоматически выполняют защиту от SQL инъекций;
- запросы, использующие `PreparedStatement`, выглядят значительно проще.

Поэтому рекомендуется всегда использовать **Prepared-Statement**.

Если вы выполните сейчас наше приложение, то обнаружите в таблице вставленные строки. Чтобы эти строки не добавлялись каждый раз при запуске приложения, закомментируйте вызовы метода `execute()`, выполняющие запросы `insert`. Теперь перед нами стоит задача прочитать данные из таблицы и вывести их в консольное окно. Для этого мы будем работать с типом **ResultSet**. Это аналог `SqlDataReader` в ADO.NET. **ResultSet** надо использовать при выполнении SQL запросов `select`. Данные, возвращаемые запросом `select`, вставляются в тип **ResultSet**, откуда их можно затем извлекать в цикле. Обратите внимание на методы `getString()`, `getDouble()` и другие аналогичные, позволяющие извлекать прочитанные данные согласно их типу. Мы передаем этим методам строковые имена полей из таблицы, однако мы могли бы указывать порядковые номера требуемых полей (нумерация начинается с 1):

```
String sel="select * from Student";
ResultSet set=db.getSelectQuery(sel, conn);
while(set.next()){
    System.out.println(set.getInt("id")+" "+
        set.getString("name")+" "+
        set.getDouble("rate"));
}
```

Добавьте этот код и запустите приложение. У меня вывод получился таким:

```
1 Corwin of Amber 190.0
2 Han Solo 180.0
```

Как видите, данные из нашей формы прочитаны и выведены в консольное окно. Обратите внимание на блок **finally**, в котором выполняется закрытие созданного соединения.

Для начала знакомства с JDBC этого примера достаточно. Далее в уроке мы еще рассмотрим примеры использования JDBC, но уже в веб приложении. Оставьте созданную базу данных **jtest**, мы еще будем с ней работать.

Пример использования Hibernate

Теперь мы можем рассмотреть пример использования **Hibernate**. В этом примере рассмотрим подключение к БД, настройку ORM отображения между таблицей БД и классом в приложении и реализуем добавление данных в таблицу. Для создания пользовательского интерфейса используем еще одну важную компоненту Java EE — *Java Server Faces (JSF)*.

JSF — это MVC веб-фреймворк, предназначенный для создания пользовательского интерфейса в веб-приложениях. Этот фреймворк предоставляет пользователю набор удобных, повторно используемых компонент для веб-страницы. JSF создает связь таких компонент (которые часто называют виджетами) с источниками данных, с одной стороны, и с событиями на стороне сервера — с другой. JSF значительно упрощает разработку пользовательского интерфейса и получил широкое распространение в современных веб-приложениях.

Создайте в Netbeans новое веб-приложение с именем **MyHibernate**. Выберите в качестве сервера **Tomcat** (хотя можно выбрать и другой сервер приложений, например, **GlassFish**). В следующем окне, после выбора сервера,

отметьте опции **Java Server Faces** и **Hibernate**, а в поле **Database Connection** немного ниже выберите опцию **New Database Connection** (рис. 28).

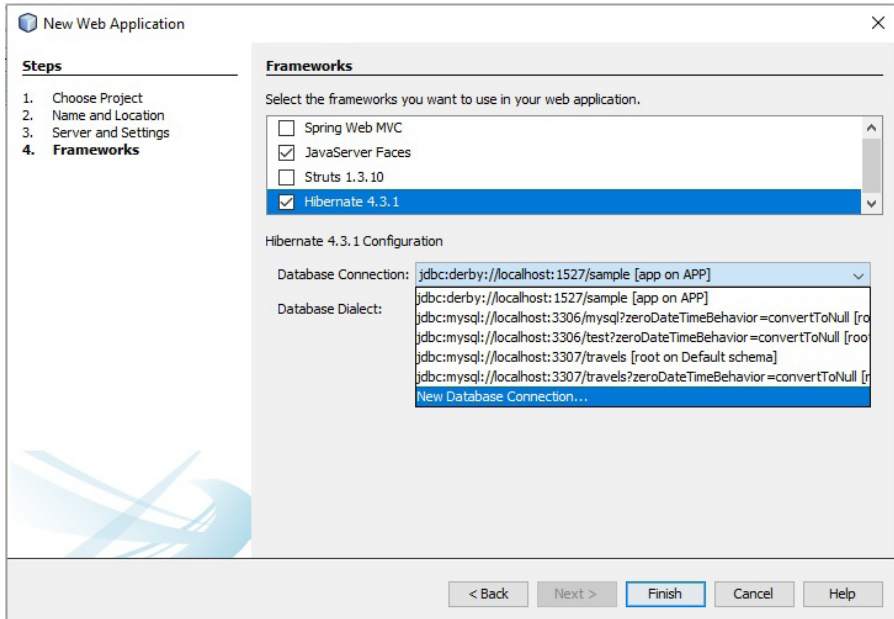


Рис 28. Создание веб-приложения с Hibernate

В следующем окне нажмите кнопку **Add** и добавьте в состав проекта JDBC драйвер для MySQL. В этом приложении мы поработаем с БД **jtest**, созданной ранее в этом уроке (рис. 29).

После выбора драйвера нажмите кнопку **Next** и перейдите к следующему окну, в котором укажите все необходимые данные для подключения к базе данных **jtest**. Обратите внимание на правильность номера порта для доступа к БД, на имя и пароль пользователя для доступа к серверу. После указания всех данных будет полезно

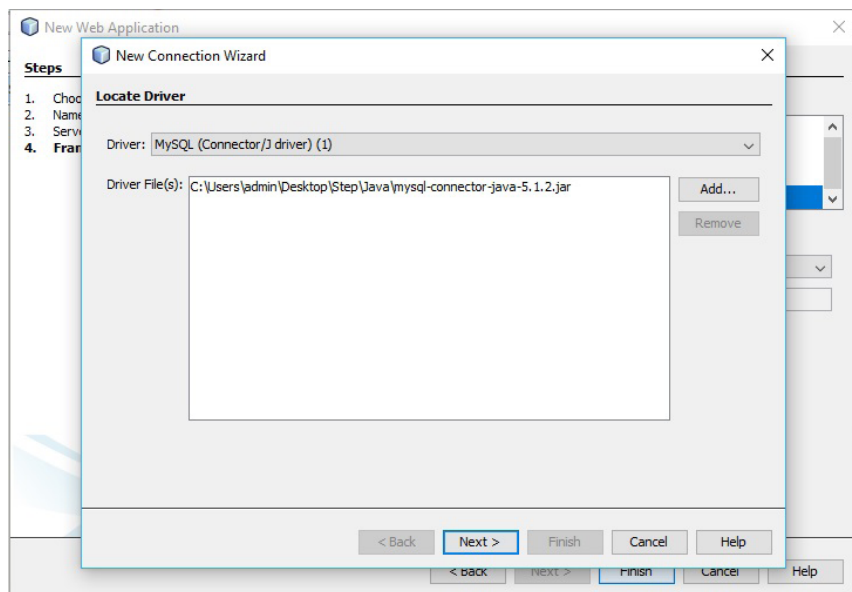


Рис 29. Добавление JDBC драйвера

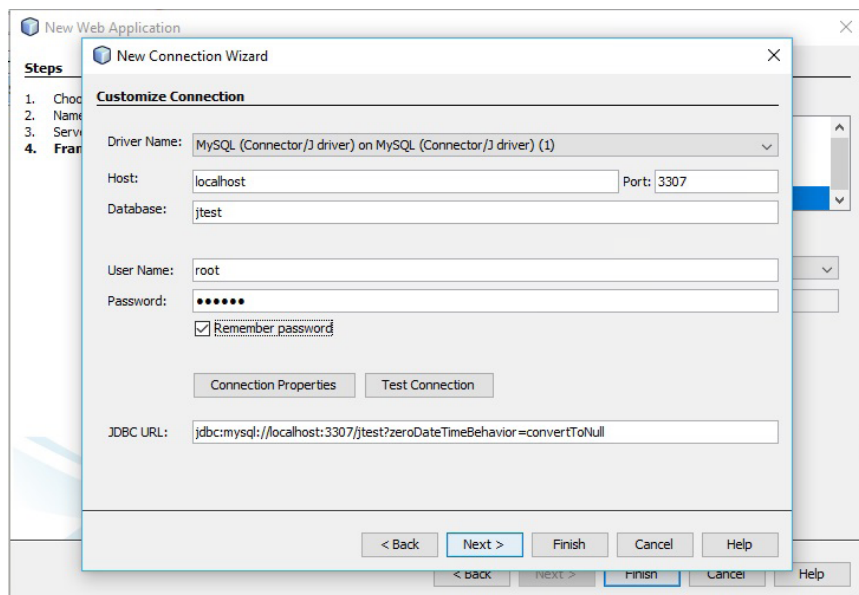


Рис 30. Создание соединения с БД

нажать кнопку **Test Connection** и проверить созданное соединение (рис. 30).

После заполнения этого окна можете нажимать **Next**, **Next** и **Finish**. Обратите внимание, что при переходе к следующему этапу создания приложения Netbeans может затормозиться. Это нормально. Надо просто немного подождать, чтобы были найдены и подгружены в проект все необходимые компоненты. Если Netbeans сообщит о проблеме по причине отсутствия драйвера JDBC, нажмите кнопку **Resolve** и укажите еще раз путь к файлу **mysql-connector-java-5.1.42-bin.jar**.

Когда проект будет создан, Netbeans откроет вам его конфигурационный файл с именем **hibernate.cfg.xml**. Запомните, этот файл обязательно должен располагаться в папке **src** созданного проекта, поэтому не надо его переносить в какие-либо пакеты. И переименовывать его не надо. В файле сейчас описано подключение к БД в соответствии с данными, указанными при создании проекта:

```
<hibernate-configuration>
<session-factory>
  <property name="hibernate.dialect">
    org.hibernate.dialect.MySQLDialect
  </property>
  <property name="hibernate.connection.driver_class">
    com.mysql.jdbc.Driver
  </property>
  <property name="hibernate.connection.url">
    jdbc:mysql://localhost:3307/
    jtest?zeroDateTimeBehavior=
    convertToNull
  </property>
  <property name="hibernate.connection.username">
```

```

        root
    </property>
    <property name="hibernate.connection.password">
        *****
    </property>
</session-factory>
</hibernate-configuration>

```

Изменять в этом файле мы пока ничего не будем, поэтому можете просто закрыть его.

Запустите созданную заготовку проекта, и вы увидите в браузере приветственную фразу *Hello from Facelets*. Эту фразу по умолчанию выводит JSF. Чтобы увидеть происхождение этого приветствия, откройте файл [index.xhtml](#), расположенный в узле проекта Web Pages. Это стартовая страница серверной части приложения. Она представляет собой xml файл, в котором располагается HTML разметка, представленная тегами JSF. Эти теги определены в пространстве имен с префиксом **h**.

```

<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0
    Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-
    transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html">
    <h:head>
        <title>Facelet Title</title>
    </h:head>
    <h:body>
        Hello from Facelets
    </h:body>
</html>

```

В этом файле мы с вами будем создавать форму для нашего приложения, но это будет немного позже.

Создайте в составе проекта новый пакет, например, с именем `myclasses`, в котором нам надо будет создавать свои классы. Выделите созданный пакет, выберите опции **New — Other — Hibernate — HibernateUtil.java**:

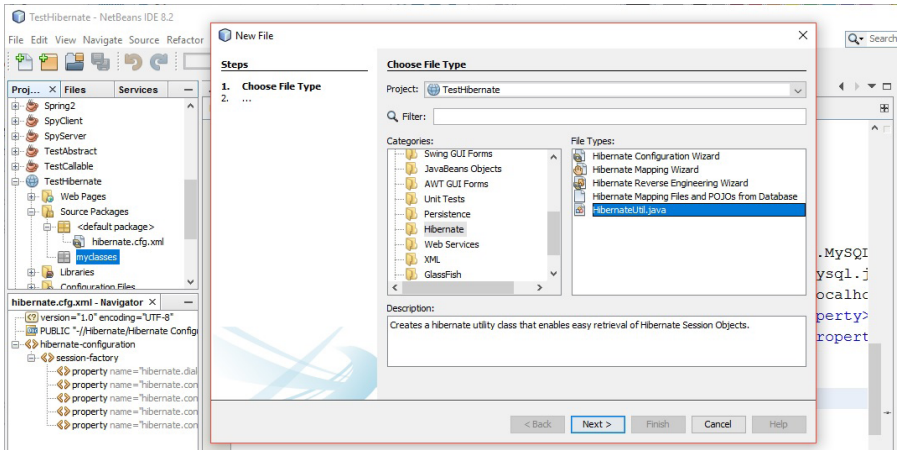


Рис 31. Создание `HibernateUtil.java`

При добавлении этого класса можете указать ему произвольное имя. Обычно его называют `HibernateUtil.java`. После добавления откройте этот класс и обязательно проверьте, не перечеркнуты ли в приведенном коде тип `AnnotationConfiguration` и метод `buildSessionFactory()`. Перечеркивание говорит о том, что данные инструменты уже устарели, а ваш `Netbeans` еще работает по старой схеме. В этом нет ничего страшного. Просто приведите код класса `HibernateUtil.java` к такому виду:

```
package myclasses;
import org.hibernate.SessionFactory;
```

```

import org.hibernate.boot.registry.
StandardServiceRegistryBuilder;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;

public class HibernateUtil {
    private static SessionFactory sessionFactory;
    public static SessionFactory getSessionFactory() {
        if (sessionFactory == null) {
            // loads configuration and mappings
            Configuration configuration =
                new Configuration().configure();
            ServiceRegistry serviceRegistry =
                new StandardServiceRegistryBuilder().
                    applySettings(configuration.
                        getProperties()).build();
            //builds a session factory from the service
            //registry
            sessionFactory = configuration.
                buildSessionFactory(serviceRegistry);
        }
        return sessionFactory; //return sessionFactory;
    }
}

```

Задача этого класса заключается в создании объекта **SessionFactory**. Этот объект создается на основании файла конфигурации **hibernate.cfg.xml** и предназначается для обслуживания общения клиента и сервера.

В базе данных **jtest**, к которой мы подключились в этом проекте, есть единственная таблица **Student** с полями **id**, **name** и **rate**. Для создания *объектно-реляционного отображения* (ORM) между нашим приложением и базой данных **jtest** мы создадим в приложении класс, соответствующий таблице **Student**.

Снова выделите в составе проекта пакет `myclasses` и добавьте в него класс `Student`. В этом классе надо создать поля, совпадающие по типу и именам с полями таблицы `Student`. Не забудьте оформить класс как `Java bean`. Приведите добавленный класс к такому виду:

```
public class Student {
    private int id;
    private String name;
    private double rate;

    public Student(String name, double rate){
        this.name=name;
        this.rate=rate;
    }

    public Student(){
        this.name="";
        this.rate=0;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

```

public double getRate() {
    return rate;
}

public void setRate(double rate) {
    this.rate = rate;
}
}

```

Теперь нам надо объяснить **Hibernate**, что он должен связать таблицу БД **Student** с созданным классом **Student**. Для этого мы создадим в этом же пакете специальный xml-файл. Снова выделите пакет **myclasses** и выберите опции **New — Other — Hibernate — Hibernate Mapping Wizard**:

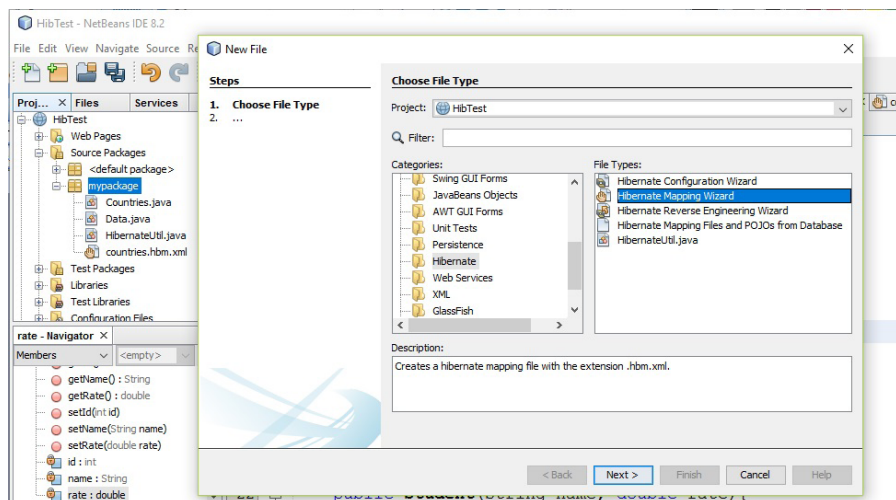


Рис 32. Выбор Hibernate Mapping Wizard

Укажите имя для создаваемого файла **student.hbm** (hbm — *Hibernate mapping*) и перейдите к следующе-

му окну, в котором укажите, что надо установить связь между нашим классом **Student** и таблицей **Student**. Будьте внимательны при вводе имени класса. Если в других загруженных проектах **Netbeans** есть классы с именем **Student**, вы увидите их в этом окне. Поэтому выберите именно класс **Student** из текущего проекта.

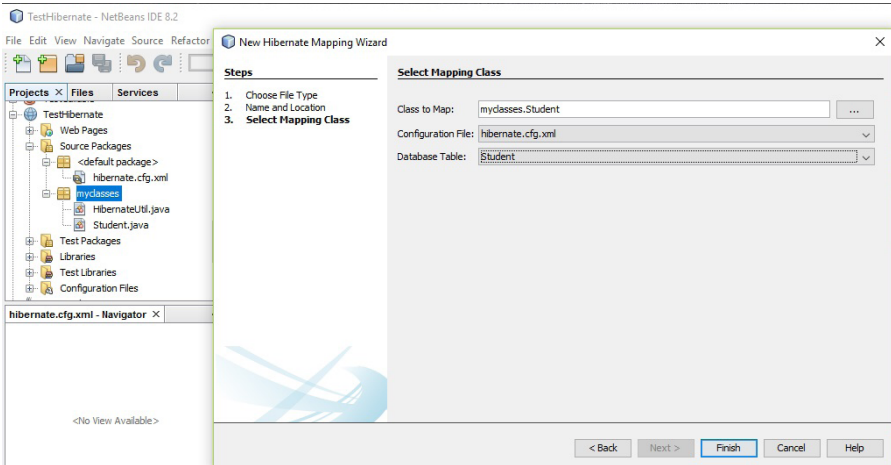


Рис 33. Запуск Hibernate Mapping Wizard

Откройте созданный файл **student.hbm.xml** и приведите его разметку к такому виду:

```
<hibernate-mapping>
  <class catalog="jtest" name="myclasses.Student"
    table="student">
    <id column="id" name="id" type="java.lang.Integer">
      <generator class="identity"/>
    </id>
    <property column="name" name="name" type="string"/>
    <property column="rate" name="rate" type="double"/>
  </class>
</hibernate-mapping>
```

Описание довольно простое. Мы указываем описание класса в парном элементе `class`. Обратите внимание, что имя класса указывается с именем пакета. Затем внутри описываем первичный ключ `id` и, в элементах `property`, описываем свойства нашего класса.

Теперь снова посмотрите в файл конфигурации `hibernate.cfg.xml` и убедитесь, что в него была добавлена еще одна строка с элементом `mapping`, указывающая на созданное нами соответствие:

```
<mapping resource="myclasses/student.hbm.xml"/>
```

На этом этапе можно сказать, что все настройки выполнены и мы можем переходить к программированию. Сейчас мы создадим класс, в котором будет описано взаимодействие классов приложения с БД. Этот класс тоже надо оформить как Java bean. Имя у него может быть произвольным. В нашем случае назовем его `StudentDb`. Приведите код этого класса к такому виду:

```
import javax.faces.bean.ManagedBean;
import myclasses.Student;
import javax.faces.bean.SessionScoped;
import org.hibernate.Session;

@ManagedBean
@SessionScoped
public class StudentDb {
    private Student s;
    private HibernateUtil helper;
    private Session session;
    private int id;
    private String name;
```

```

private double rate;
public StudentDb() {}
public void addStudent() {
    if (this.name != "") {
        s.setName(this.name);
        s.setRate(this.rate);
        session = helper.getSessionFactory().
            openSession();
        session.beginTransaction();
        session.save(s);
        session.getTransaction().commit();
        session.close();
    }
}

public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public double getRate() {
    return rate;
}

public void setRate(double rate) {
    this.rate = rate;
}
}

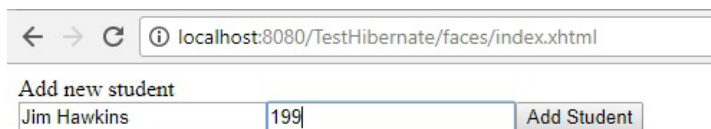
```

Этот класс декорирован обязательными аннотациями. Будьте внимательны при импорте пакетов для этих аннотаций и для объекта `Session`, поскольку существуют типы с такими же именами, но описанные в других пакетах. В этом классе особое значение для нас имеет метод `addStudent()`. Именно он будет обработчиком формы, которую мы сейчас создадим на главной странице. В этом методе происходит подключение к БД через объект `Session`, затем создается объект `Student`, который инициализируется значениями, взятыми из формы, и заносится в БД. Обратите внимание, что работа с БД оформлена в виде транзакции.

Теперь нам осталось добавить форму в файл `index.xhtml`. Откройте его и добавьте такую разметку в элемент `<h:body>`:

```
<h:body>
  <h:outputLabel value="Add new student"/>
  <h:form>
    <h:inputText value="#{studentDb.name}" />
    <h:inputText value="#{studentDb.rate}" />
    <h:commandButton value="Add Student"
      action="#{data.addStudent()}" />
  </h:form>
</h:body>
```

Запустите созданное приложение. В этот раз вместо приветственной фразы вы увидите нашу простую форму. Заполните ее и нажмите кнопку:



The screenshot shows a web browser window with the address bar displaying `localhost:8080/TestHibernate/faces/index.xhtml`. Below the address bar, the page title is "Add new student". The form contains two text input fields. The first field has the text "Jim Hawkins" and the second field has the text "199". To the right of the second input field is a button labeled "Add Student".

Рис 34. Запуск приложения

После этого запустите снова [phpmyadmin](#), перейдите в базу данных [jtest](#) и убедитесь, что в таблицу [Student](#) была добавления новая запись:

				id	name	rate
☐				1	Corwin of Amber	190
☐				2	Han Solo	180
☐				3	Jim Hawkins	199

Рис 35. Новая запись в таблице Student

Если после нажатия на кнопку в нашей форме приложение выбросило исключение [org.hibernate.exception.JDBCConnectionException](#), выполните еще раз добавление к проекту драйвера JDBC.

Домашнее задание

В качестве домашнего задания вам предлагается создать веб-приложение. Это приложение должно использовать сервлеты, которые, в свою очередь, должны работать с нашей базой данных `jtest`.

На главной странице приложения создайте форму с кнопкой `Add` для добавления новых записей в таблицу `Student`, подобно тому, как это было сделано в последнем приложении. Кроме этого, на главной странице создайте еще одну кнопку `Show`. При нажатии на эту кнопку на странице должны отображаться все записи из таблицы `Student`.

Обработчиками кнопок `Add` и `Show` должны быть сервлеты. Это могут быть два разных сервлета или один и тот же. Формат вывода данных из таблицы придумайте самостоятельно.

